

Module 2

- **Understanding Requirements**
- **Requirement Modelling : Scenarios, Information & Analysis classes**
- **Requirement Modeling: Flow, Behavior**

Requirements Engineering

- Many software developers want to jump right in building software before they have a clear understanding of what is needed.
- They argue that things will become clear as they build, that project stakeholders will be able to understand need only after examining early iterations of the software,
- Things change so rapidly that any attempt to understand requirements in details is a waste of time.
- The bottom line is producing a working program and all else is secondary.
- What makes these arguments seductive is that they contain elements of truth. But each is flawed and can lead to a failed software project.
- *The broad spectrum of tasks and techniques that lead to an understanding of requirements is called requirements engineering.* It begins with communication activity and continues into the modeling activity. It builds a bridge to design and construction.
- Requirements engineering provides the appropriate mechanism for understanding what the customer wants, analyzing need, assessing feasibility, negotiating a reasonable solution, specifying the solution unambiguously, validating the specification, and managing the requirements as they are transformed into an operational system

5.1 Requirements Engineering

It encompasses seven tasks. Some tasks can occur in parallel.

1. Inception — ... (Understanding the problem)

- basic understanding of the problem
- the people who want a solution
- the nature of the solution that is desired, and
- the effectiveness of preliminary communication and collaboration between the customer and the developer

2. Elicitation — ask customers, users what the objectives for the system or product are, , what is to be accomplished, how the system fits into the needs of the business and how the system is to be used on a day-today basis. elicit requirements from all stakeholders. A number of problems that are encountered as elicitation occurs.

- **Problems of scope.: The boundary of the system is ill-defined or the** customers/users specify unnecessary technical detail that may confuse, rather than clarify, overall system objectives.
- **Problems of understanding: The customers/users are not completely sure** of what is needed, have a poor understanding of the capabilities and limitations of their computing environment, don't have a full understanding of the problem domain, have trouble communicating needs to the system engineer,
- **Problems of volatility: The requirements change over time.**

Requirements Engineering.....

3. Elaboration—

- the information obtained from the customer during inception and elicitation is expanded and refined during elaboration.
- This task focuses on developing *a refined requirements model* that identifies various aspects of software function, behavior, and information.
- Elaboration is driven by the creation and refinement of user scenarios that describe how the end user (and other actors) will interact with the system.

Requirements Engineering...

4. **Negotiation**—You have to reconcile through a process of negotiation.

- Using an iterative approach that prioritizes requirements, assesses their cost and risk, and addresses internal conflicts,
- requirements are eliminated, combined, and/or modified so that each party achieves some measure of satisfaction.

5. **Specification**—can be any one (or more) of the following:

- A written document
- A set of graphical models
- A formal mathematical model
- A collection of user scenarios (use-cases)
- A prototype
- Combination of all



Software Requirements Specification Template

A software requirements specification (SRS) is a document that is created when a detailed description of all aspects of the software to be built must be specified before the project is to commence. It is important to note that a formal SRS is not always written. In fact, there are many instances in which effort expended on an SRS might be better spent in other software engineering activities. However, when software is to be developed by a third party, when a lack of specification would create severe business issues, or when a system is extremely complex or business critical, an SRS may be justified.

Karl Wiegiers [Wie03] of Process Impact Inc. has developed a worthwhile template (available at www.processimpact.com/process_assets/srs_template.doc) that can serve as a guideline for those who must create a complete SRS. A topic outline follows:

Table of Contents Revision History

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Project Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective

- 2.2 Product Features
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. System Features

- 3.1 System Feature 1
- 3.2 System Feature 2 (and so on)

4. External Interface Requirements

- 4.1 User Interfaces
- 4.2 Hardware Interfaces
- 4.3 Software Interfaces
- 4.4 Communications Interfaces

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes

6. Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

Appendix C: Issues List

A detailed description of each SRS topic can be obtained by downloading the SRS template at the URL noted earlier in this sidebar.

Requirements Engineering...

6. **Validation**—a review mechanism that looks for review team to find
- errors in content or interpretation
 - areas where clarification may be required
 - missing information
 - inconsistencies (a major problem when large products or systems are engineered)
 - conflicting or unrealistic (unachievable) requirements.
7. **Requirements management**—
- Requirements change and the desire to change persists throughout the life of the system.
 - Requirements management is a set of activities that help the project team *identify, control and track* requirements and changes to requirements at any time as the project proceeds.

5.2 Establishing the groundwork

1. Identify stakeholders (who benefit directly or indirectly from the system)

- Suggested List and will grow: Business operations managers, product managers, marketing people, customers, end users, consultants, product engineers, support engineer, software engineer.
- Each stakeholder has a different view of the system, different benefits if project succeeds and open to risk if project fails
- “who else do you think I should talk to?” is asked for every stakeholder

2. Recognize multiple points of view and category

- As information from multiple viewpoints is collected, emerging requirements may be inconsistent or may conflict with one another.
- Categorize all stakeholder information (including inconsistent and conflicting requirements) in a way that will allow decision makers to choose an internally consistent set of requirements for the system.

5.2 Establishing the groundwork

3. **Work toward collaboration**

- If five stakeholders are involved in a software project, it may have five (or more) different opinions about the proper set of requirements.
- Customers (and other stakeholders) must collaborate among themselves and with software engineering practitioners if a successful system is to result. But how is this collaboration accomplished?
- The job of a requirements engineer is to identify areas of commonality (i.e., requirements on which all stakeholders agree) and areas of conflict or inconsistency (i.e., requirements that are desired by one stakeholder but conflict with the needs of another stakeholder.)
- a strong “project champion”(e.g., a business manager or a senior technologist) may make the final decision about which requirements make the cut.

5.2 Establishing the groundwork

4. Asking the First Questions:

The first set of the **context-free questions** focuses on the customer and other stakeholders, *the overall project goals and benefits*.

- Who is behind the request for this work?
- Who will use solution?
- What will be the economic benefit of a successful solution
- Is there another source for the solution that you need?

The next set of questions enables you to **gain a better understanding of the problem** and allow **customers to voice their perceptions about solutions**

- How would you characterize “good” output that would be generated by a successful solution?
- What problems will this solution address?
- Can you show or describe me the business environment in which solution will be used?
- Will special performance issues or constraints affect the way the solution is approached?

5.2 Establishing the groundwork

4. Asking the First Questions (Conti..)

The final set of questions focuses on the effectiveness of the communication activity itself.

- Are you the right person to answer these questions? Are your answers official?
- Are my questions relevant to the problem that you have?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

5.3 Eliciting or Gathering Requirements

• Requirements elicitation combines elements of problem solving, elaboration, negotiation and specification.

1. Collaborative Requirements Gathering

Some basic guidelines for productive collaborative work:

- meetings are conducted and attended by both software engineers and customers
- rules for preparation and participation are established
- an agenda is suggested (formal enough and be flexible)
- a "facilitator" (can be a customer, a developer, or an outsider) controls the meeting
- a "definition mechanism" (can be work sheets, flip charts, or wall stickers or an electronic bulletin board, chat room or virtual forum) is used
- the goal is
 - to identify the problem
 - propose elements of the solution
 - negotiate different approaches, and
 - specify a preliminary set of solution requirements in an atmosphere that is conducive to the accomplishment of the goals.

Conti...

the sequence of events that lead up to the requirements gathering meeting, occur during the meeting, and follow the meeting.

- During inception basic questions and answers establish the scope of the problem and the overall perception of a solution. Out of these initial meetings, the developer and customers write a one- or two-page “**product request.**”
- A meeting place, time, and date are selected; a facilitator is chosen; and attendees from the software team and other stakeholder organizations are invited to participate. The product request is distributed to all attendees before the meeting date.

Example of Product Request: About home security function that is a part of “SAFE HOME”

Our research indicates that the market for home management systems is growing at a rate of 40 percent per year. The first *SafeHome* function we bring to market should be the home security function. Most people are familiar with “alarm systems” so this would be an easy sell.

The home security function would protect against and/or recognize a variety of undesirable “situations” such as illegal entry, fire, flooding, carbon monoxide levels, and others. It’ll use our wireless sensors to detect each situation. It can be programmed by the homeowner, and will automatically telephone a monitoring agency when a situation is detected.

- While reviewing the product request in the days before the meeting, each attendee is asked to make a list of
 1. objects that are part of the environment that surrounds the system, other objects that are to be produced by the system, and objects that are used by the system to perform its functions.
 2. each attendee is asked to make another list of services (processes or functions) that manipulate or interact with the objects.
 3. lists of constraints (e.g., cost, size, business rules)
 4. performance criteria (e.g., speed, accuracy) are also developed.



- The lists of objects are displayed using various means. Ideally, each listed entry should be capable of being manipulated separately so that lists can be combined, entries can be modified, and additions can be made. At this stage, critique and debate are strictly prohibited.
- After individual lists are presented in one topic area, the group creates a combined list by eliminating redundant entries, adding any new ideas that come up during the discussion, but not deleting anything.
- The combined list is shortened, lengthened, or reworded to properly reflect the product/system to be developed.
- stakeholders develop *mini-specifications for entries on* the lists. Each mini-specification is an elaboration of an object or service

The control panel is a wall-mounted unit that is approximately 9×5 inches in size. The control panel has wireless connectivity to sensors and a PC. User interaction occurs through a keypad containing 12 keys. A 3×3 inch LCD color display provides user feedback. Software provides interactive prompts, echo, and similar functions.

- The mini-specs are presented to all stakeholders for discussion.
- During all discussions, the team may raise an issue that cannot be resolved during the meeting. *An issues list is maintained so that these ideas will be acted on later.*



Conducting a Requirements Gathering Meeting

The scene: A meeting room. The first requirements gathering meeting is in progress.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator (pointing at whiteboard): So that's the current list of objects and services for the home security function.

Marketing person: That about covers it from our point of view.

Vinod: Didn't someone mention that they wanted all *SafeHome* functionality to be accessible via the Internet? That would include the home security function, no?

Marketing person: Yes, that's right . . . we'll have to add that functionality and the appropriate objects.

Facilitator: Does that also add some constraints?

Jamie: It does, both technical and legal.

Production rep: Meaning?

Jamie: We better make sure an outsider can't hack into the system, disarm it, and rob the place or worse. Heavy liability on our part.

Doug: Very true.

Marketing: But we still need that . . . just be sure to stop an outsider from getting in.

Ed: That's easier said than done and . . .

Facilitator (interrupting): I don't want to debate this issue now. Let's note it as an action item and proceed.

(Doug, serving as the recorder for the meeting, makes an appropriate note.)

Facilitator: I have a feeling there's still more to consider here.

(The group spends the next 20 minutes refining and expanding the details of the home security function.)

Eliciting or Gathering Requirements

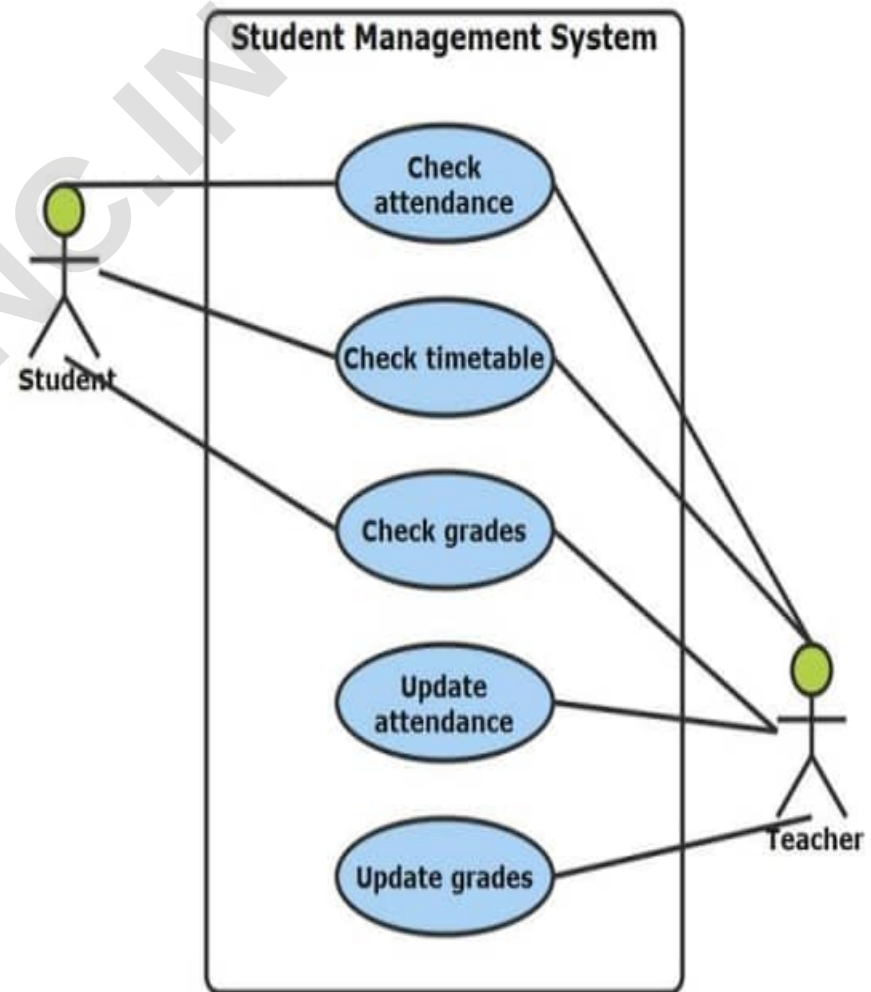
2. Quality Function Deployment(QFD)

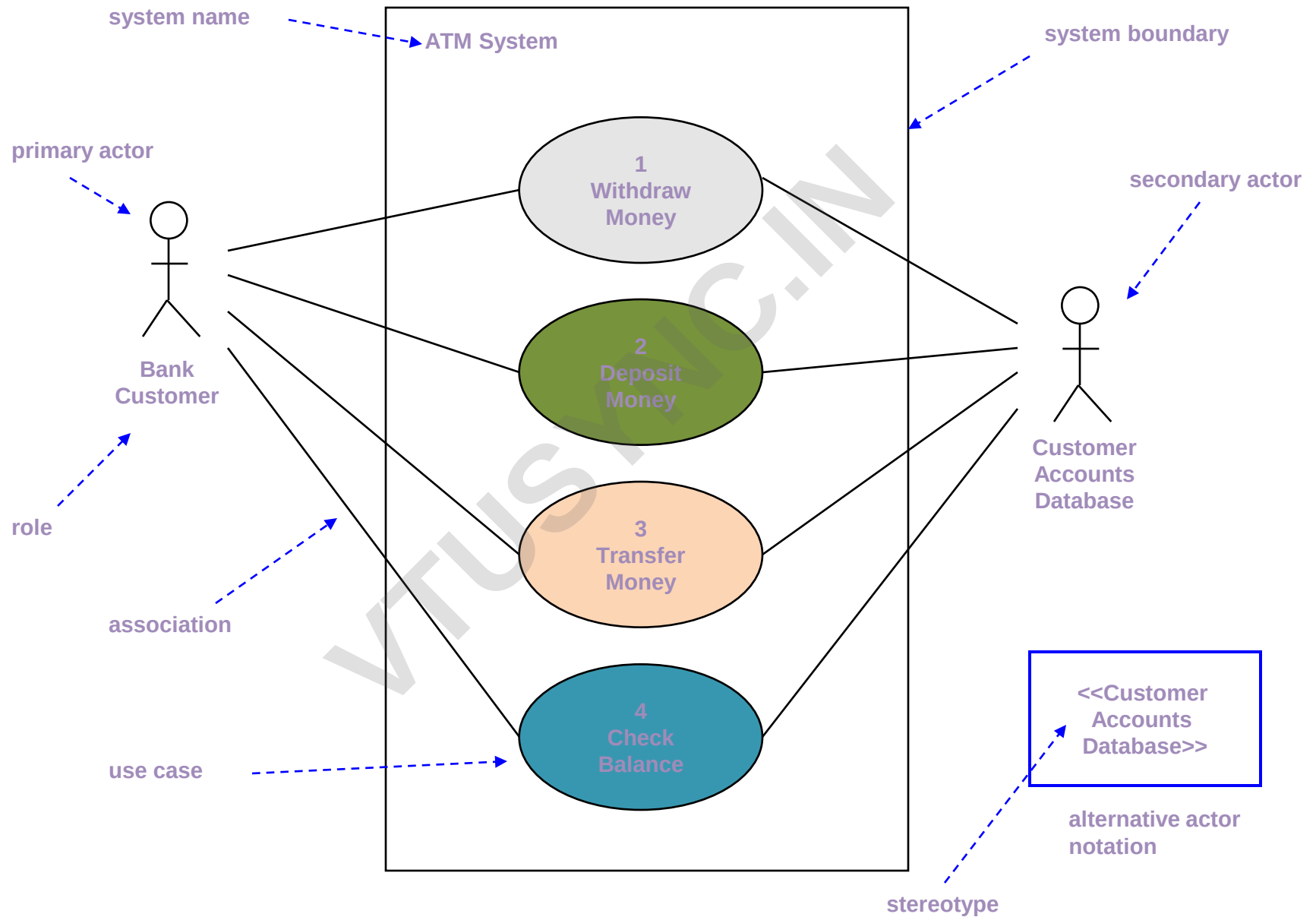
- QFD is a quality management technique that translates the needs of the customer into technical requirements for software.
- It concentrates on maximizing customer satisfaction from the software engineering process.
- It identifies three types of requirements.
 - **Normal requirements.**
The objectives and goals that are stated for a product or system during meetings with the customer. If these requirements are present, the customer is satisfied.
Examples: requested types of graphical displays, specific system functions, and defined levels of performance.
 - **Expected requirements.**
These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them. Their absence will be a cause for significant dissatisfaction.
Examples: ease of human/machine interaction, overall operational correctness and reliability, and ease of software installation.
 - **Exciting requirements:**
Features go beyond the customer's expectations and prove to be very satisfying when present.
Examples: software for a new mobile phone coupled with a set of additional capabilities, multi-touch screen, visual voice mail that delight every users of the product.
- **Function deployment** determines the “value” (as perceived by the customer) of each function required of the system and deploys these values throughout.

Eliciting or Gathering Requirements

3. Usage Scenarios

- As requirements are gathered, an overall vision of system functions and features begins to materialize.
- However, it is difficult to move into more technical software engineering activities until you understand how these functions and features will be used by different classes of end users.
- To accomplish this, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed.
- The scenarios, often called *use cases* [Jac92], provide a description of how the system will be used.





Eliciting or Gathering Requirements

4. Elicitation Work Product

- Will vary depending on the size of the system or product to be built. For most systems, the work products include:
 - A statement of need and feasibility
 - A bounded statement of scope for the system or product
 - A list of customers, users and other stakeholders who participated in requirements elicitation
 - A description of the system's technical environment
 - A list of requirements preferably organized by function and the domain constraints that apply to each
 - A set of usage scenarios that provide insight into the use of the system or product under different operating conditions.
 - Any prototypes developed to better define requirements.



Developing a Preliminary User Scenario

The scene: A meeting room, continuing the first requirements gathering meeting.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator: We've been talking about security for access to *SafeHome* functionality that will be accessible via the Internet. I'd like to try something. Let's develop a usage scenario for access to the home security function.

Jamie: How?

Facilitator: We can do it a couple of different ways, but for now, I'd like to keep things really informal. Tell us (he points at a marketing person) how you envision accessing the system.

Marketing person: Um . . . well, this is the kind of thing I'd do if I was away from home and I had to let someone into the house, say a housekeeper or repair guy, who didn't have the security code.

Facilitator (smiling): That's the reason you'd do it . . . tell me how you'd actually do this.

Marketing person: Um . . . the first thing I'd need is a PC. I'd log on to a website we'd maintain for all users of *SafeHome*. I'd provide my user id and . . .

Vinod (interrupting): The Web page would have to be secure, encrypted, to guarantee that we're safe and . . .

Facilitator (interrupting): That's good information, Vinod, but it's technical. Let's just focus on how the end user will use this capability. OK?

Vinod: No problem.

Marketing person: So as I was saying, I'd log on to a website and provide my user ID and two levels of passwords.

Jamie: What if I forget my password?

Facilitator (interrupting): Good point, Jamie, but let's not address that now. We'll make a note of that and call it an *exception*. I'm sure there'll be others.

Marketing person: After I enter the passwords, a screen representing all *SafeHome* functions will appear. I'd select the home security function. The system might request that I verify who I am, say, by asking for my address or phone number or something. It would then display a picture of the security system control panel

along with a list of functions that I can perform—arm the system, disarm the system, disarm one or more sensors. I suppose it might also allow me to reconfigure security zones and other things like that, but I'm not sure.

(As the marketing person continues talking, Doug takes copious notes; these form the basis for the first informal usage scenario. Alternatively, the marketing person could have been asked to write the scenario, but this would be done outside the meeting.)

5.4 Developing Use Cases

- “a use case captures a contract ... [that] describes the system’s behavior under various conditions as the system responds to a request from one of its stakeholders . . .”
- In essence, a use case tells a stylized story about how an end user interacts with the system under a specific set of circumstances. The story can be a narrative text, an outline of tasks or interactions, a template-based description, or a diagrammatic representation.
- The first step in writing a use case is to define the set of “actors” that will be involved in the story
- *Actors are the different people (or devices) that use the system or product within the context of the function and behavior that is to be described*
- an actor is anything that communicates with the system or product and that is external to the system itself. Every actor has one or more goals when using the system.
- It is important to note that an actor and an end user are not necessarily the same thing.

5.4 Developing Use-Cases

- A typical user may play a number of different roles when using a system, whereas an actor represents a class of external entities (often, but not always, people) that play just one role in the context of the use case.
- **For example, a machine** in four different modes of programming, test, monitoring and troubleshooting modes. Four actors can be defined. Programmer, tester, monitor, and troubleshooter. In some cases, the machine operator can play all of these roles. Or different people can play different roles.
- Because requirements elicitation is an evolutionary activity, not all actors are identified during the first iteration.
- It is possible to identify primary actors [Jac92] during the first iteration and secondary actors as more is learned about the system.
- **Primary actors** *interact to achieve required system function and derive the intended benefit from the system. They work directly and frequently with the software.*
- **Secondary actors** *support the system so that primary actors can do their work*

5.4 Developing Use-Cases

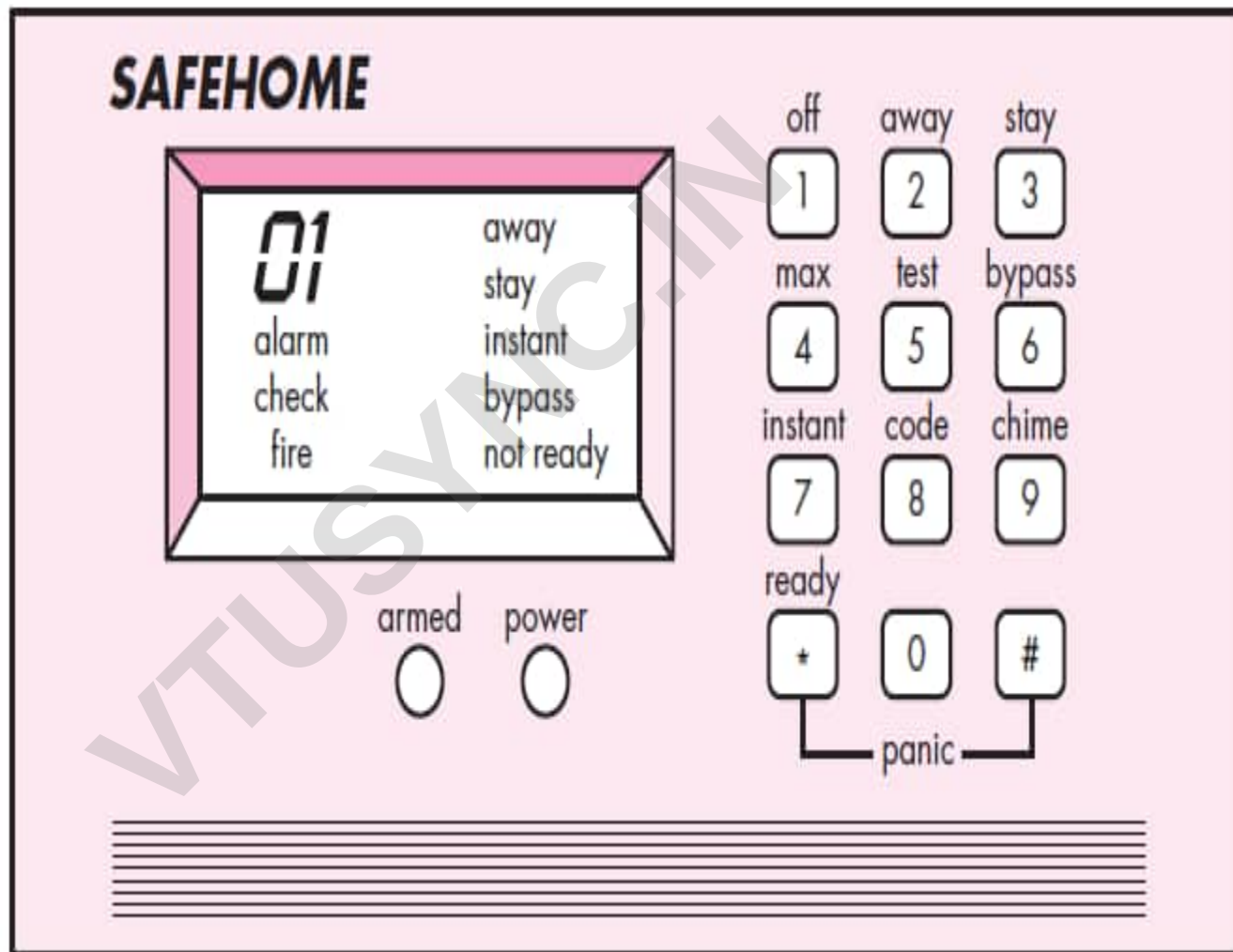
- Once actors are defined, use cases can be developed. Each scenario answers the following questions:
 - Who is the primary actor, the secondary actor (s) (to support the primary actors)?
 - What are the actor' s goals?
 - What preconditions should exist before the story begins?
 - What main tasks or functions are performed by the actor?
 - What extensions might be considered as the story is described?
 - What variations in the actor' s interaction are possible?
 - What system information will the actor acquire, produce, or change?
 - Will the actor have to inform the system about changes in the external environment?
 - What information does the actor desire from the system?
 - Does the actor wish to be informed about unexpected changes?

Use-Cases for SafeHome

- Four actors are defined:
 1. **homeowner** (a user),
 2. **setup manager** (likely the same person as homeowner but playing a different role),
 3. **sensors** (device attached to the system),
 4. **the monitoring and response subsystem** (the central station that monitors the home security function).
- To simplify the example, we only consider homeowner actor, who interacts with the home security function in a number of different ways using either the alarm control pane or a PC.
 - Enters a password to allow all other interactions
 - Inquires about the status of a security zone
 - Inquires about the status of a sensor
 - Press the panic button in an emergency
 - Activates/deactivate the security system

FIGURE 5.1

SafeHome
control panel



Use-Cases for SafeHome

Considering the situation in which the homeowner uses the control panel, the **basic use case (presenting a high-level story) for system activation** follows:

1. The homeowner observes the *SafeHome* control panel to determine if the system is ready for input. If the system is not ready, a not ready message is displayed, and the homeowner must physically close windows or doors so that not ready message disappears. (not ready means sensor is open when door window is open)
2. The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password. If it is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further actions.
3. The homeowner selects and keys in stay or away to activate the system. Stay activates only perimeter sensors (inside motion detecting sensor are deactivated). Away activates all sensors.
4. When activation occurs, a read alarm light can be observed by the homeowner.

Use-Cases for SafeHome

- Uses cases are further elaborated to provide more details.
- **Use case:** *InitiateMonitoring*
- **Primary actor:** Homeowner
- **Goal in context:** To set the system to monitor sensors when the homeowner leaves the house or remains inside.
- **Precondition:** system has been programmed for a password and to recognize various sensors
- **Trigger:** the homeowner decided to set the system to turn on the alarm function.
- **Scenario:**
 1. homeowner observes control panel,
 2. enter password,
 3. select stay or away,
 4. observes red alarm light to indicate that the house has been armed.
- **Exceptions:**
 1. control panel is not ready (check door open)
 2. password is incorrect
 3. correct password is not recognized.
 4. Stay is selected
 5. Away is selected

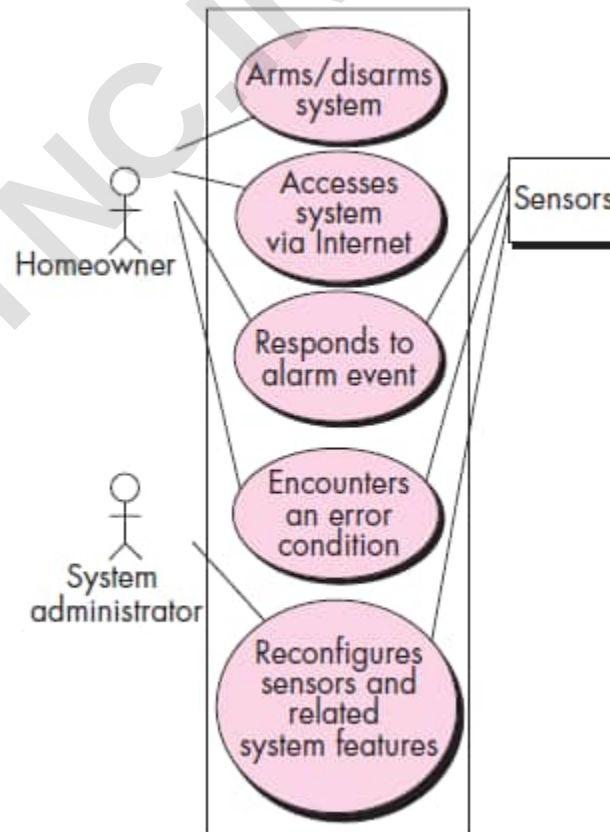
Use-Cases for SafeHome

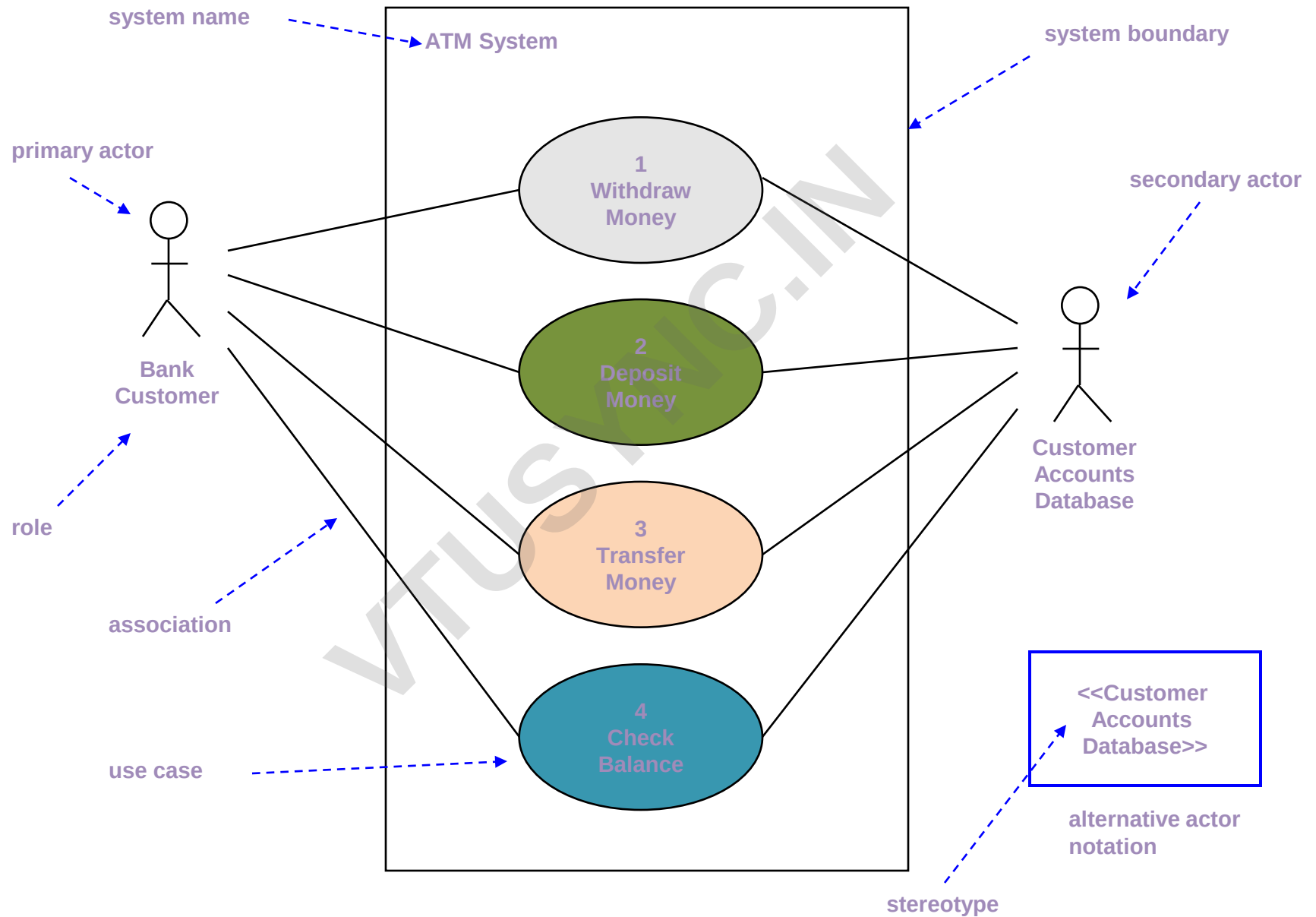
- **Priority:** Essential, must be implemented
- **When available:** First increment
- **Frequency of use:** Many times per day
- **Channel to actor:** Via control panel interface
- **Secondary actors:** Support technician, sensors
- **Channels to secondary actors:**
 - Support technician: phone line
 - Sensors: hardwired and radio frequency interfaces
- **Open issues:**
 1. Should there be a way to activate the system without the use of a password or with an abbreviated password?
 2. Should the control panel display additional text messages?
 3. How much time does the homeowner have to enter the password from the time the first key is pressed?
 4. Is there a way to deactivate the system before it actually activates?

Use-Cases for SafeHome

FIGURE 5.2

UML use case diagram for *SafeHome* home security function





5.5 Building The Requirement Models

- The intent of the analysis/requirement model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system.
- The model changes dynamically as you learn more about the system to build. Other stakeholders understand more about what they really require.
- the analysis model is a snapshot of requirements at any given time. You should expect it to change.
- As the requirements model evolves, certain elements will become relatively stable, providing a solid foundation for the design tasks that follow. However, other elements of the model may be more volatile, indicating that stakeholders do not yet fully understand requirements for the system.
- different modes of representation force you to consider requirements from different viewpoints---an approach that has a higher probability of uncovering omissions, inconsistencies, and ambiguity.
- A set of generic **Elements of the analysis model** that are common to all models.
 - Scenario-based elements
 - Functional—processing narratives for software functions
 - Use-case—descriptions of the interaction between an “actor” and the system
 - Class-based elements
 - Each usage scenario implies a set of objects (actors) that are categorized into classes. State diagram
 - Behavioral elements
 - State diagram is one method by depicting its states and the events that cause the system to change state.
 - Flow-oriented element
 - Data flow diagram, input, transform it and produce output.

5.5.1 Elements of Requirement model

- There are many different ways to look at the requirements for a computer-based system.
- Different modes of representation force you to consider requirements from different viewpoints—an approach that has a higher probability of uncovering omissions, inconsistencies, and ambiguity.
- The specific elements of the requirements model are dictated by the analysis modeling method
- A set of generic **Elements of the analysis model** that are common to all models.

1. Scenario-based elements

2. Class-based elements

3. Behavioral elements

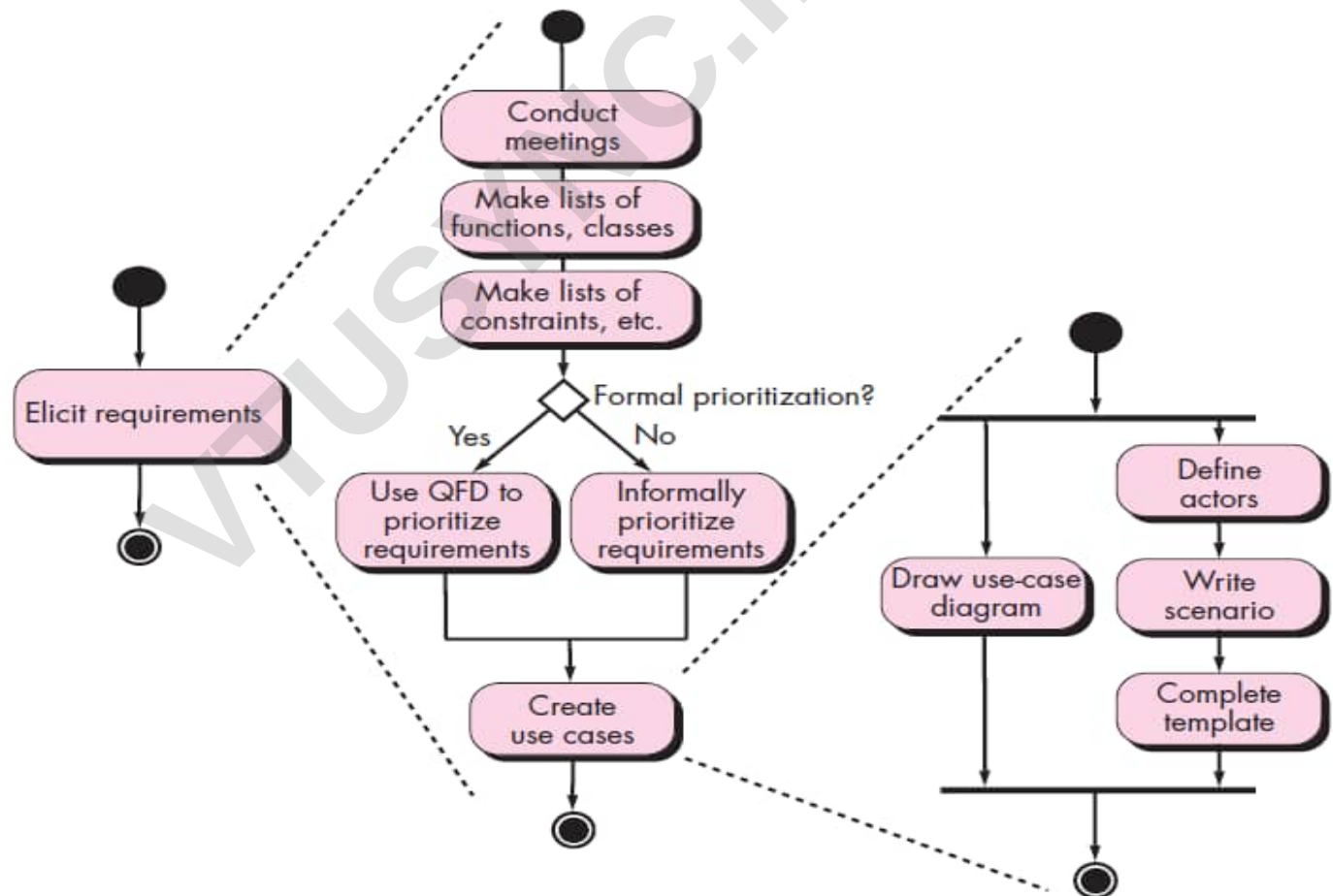
4. Flow-oriented element

5.5.1 Elements of Requirement model

1. Scenario based elements: Scenario-based elements of the requirements model are often the first part of the model that is developed. As such, they serve as input for the creation of other modeling elements

FIGURE 5.3

UML activity diagrams for eliciting requirements

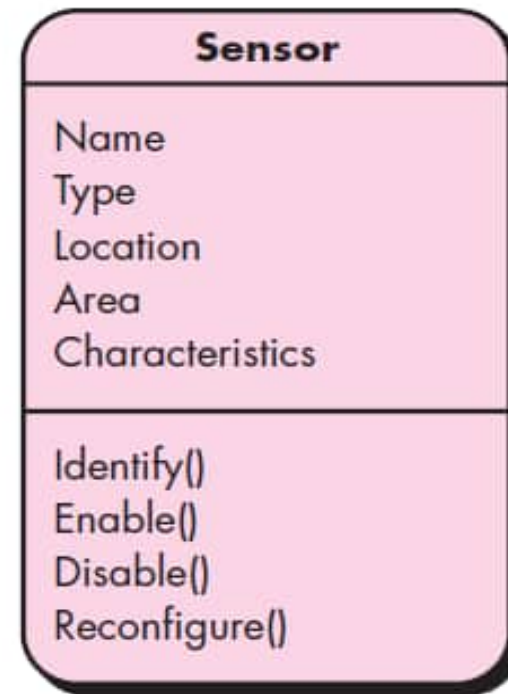


2. Class based elements:

- Each usage scenario implies a set of objects that are manipulated as an actor interacts with the system.
- These objects are categorized into classes—
tes

FIGURE 5.4

Class diagram
for sensor



3. Behavioral elements:

- The behavior of a computer-based system can have a profound effect on the design that is chosen and the implementation approach that is applied.
- Therefore, the requirements model must provide modeling elements that depict behavior.
- The *state diagram* is one method for representing the behavior of a system by depicting its states and the events that cause the system to change state.
- A *state* is any externally observable mode of behavior.
- In addition, the state diagram indicates actions (e.g., process activation) taken as a consequence of a particular event.

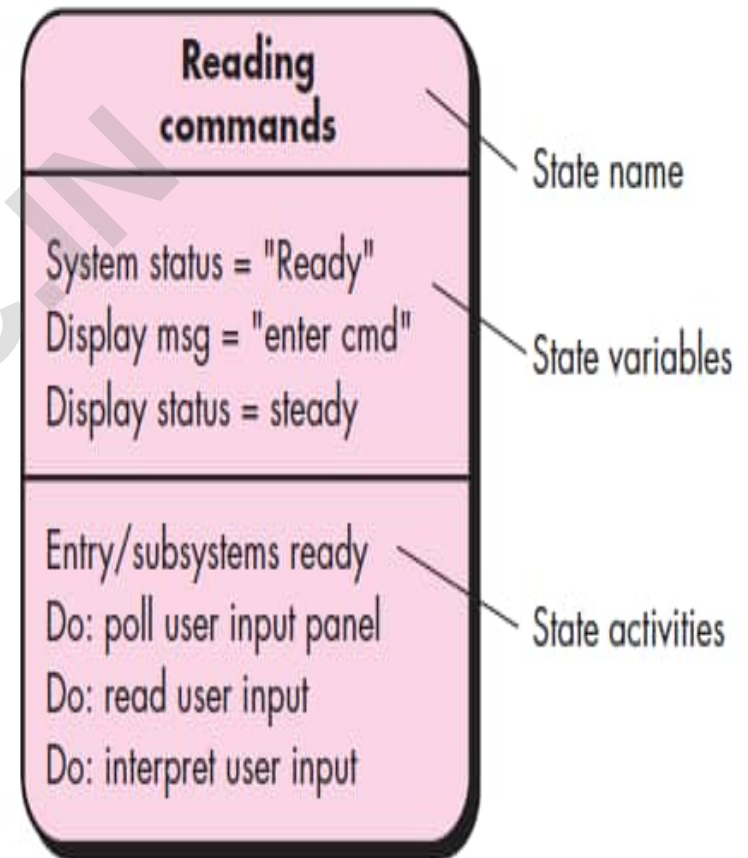


Fig: State diagram of the software embedded within control panel that is responsible for reading user input.

4. Flow-oriented elements.

- **Information is transformed as it flows through a** computer-based system.
- The system accepts input in a variety of forms, applies functions to transform it, and produces output in a variety of forms.
- Input may be a control signal transmitted by a transducer, a series of numbers typed by a human operator, a packet of information transmitted on a network link, or a voluminous data file retrieved from secondary storage.
- The transform(s) may comprise a single logical comparison, a complex numerical algorithm, or a rule-inference approach of an expert system.
- Output may light a single LED or produce a 200-page report.
- In effect, we can create a flow model for any computer-based system, regardless of size and complexity.

5.5.2 Analysis Patterns

- These *analysis patterns suggest solutions*(e.g., a class, a function, a behavior) within the application domain that can be reused when modeling many applications.
- Geyer-Schulz and Hahsler [Gey01] suggest two benefits that can be associated with the use of analysis patterns:
 1. **Analysis patterns speed up the development of abstract analysis models** that capture the main requirements of the concrete problem by providing reusable analysis models with examples as well as a description of advantages and limitations.
 2. **Analysis patterns facilitate the transformation of the analysis model into a design model** by suggesting design patterns and reliable solutions for common problems.
- Analysis patterns are integrated into the analysis model by reference to the pattern name.
- They are also stored in a repository so that requirements engineers can use search facilities to find and apply them.
- Information about an analysis pattern (and other types of patterns) is presented in a standard template

Negotiating Requirements

- Even after inception, elicitation and elaborations, customer requirements are still not sufficient in details to proceed to next activity.
- Stakeholders need to balance functionality, performance and other product or system characteristics against cost and time-to-market.
- Thus, negotiation is to develop a project plan that meets stakeholder needs while at the same time reflecting the real-world constraints such as time, people and budget.
- Identify the key stakeholders
 - These are the people who will be involved in the negotiation
- Determine each of the stakeholders “win conditions”
 - Win conditions are not always obvious
- Negotiate
 - Work toward a set of requirements that lead to “win-win” for all concerned including the software team.

Validating Requirements - I

- Is each requirement consistent with the overall objective for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?

Validating Requirements - II

- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function and behavior of the system to be built.
- Has the requirements model been “partitioned” in a way that exposes progressively more detailed information about the system.
- Have requirements patterns been used to simplify the requirements model. Have all patterns been properly validated? Are all patterns consistent with customer requirements?

Requirement Modelling Scenarios, Information & Analysis classes

6.1 Requirement Analysis

- Requirements analysis results in
 1. Specification of **software's operational characteristics**
 2. indicates **software's interface** with other system elements
 3. establishes **constraints** that software must meet
- Requirements analysis allows you (regardless of whether you're called a *software engineer, an analyst, or a modeler*) to:
 1. **elaborate** on basic requirements established during earlier requirement engineering tasks

Requirement Analysis

- The requirements modeling action results in one or more of the following types of models:
 1. *Scenario-based models* of requirements from the point of view of various system “actors”
 2. *Data models* that depict the information domain for the problem
 3. *Class-oriented models* that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements
 4. *Flow-oriented models* that represent the functional elements of the system and how they transform data as it moves through the system
 5. *Behavioral models* that depict how the software behaves as a consequence of external “events”

Overall objectives and Philosophy

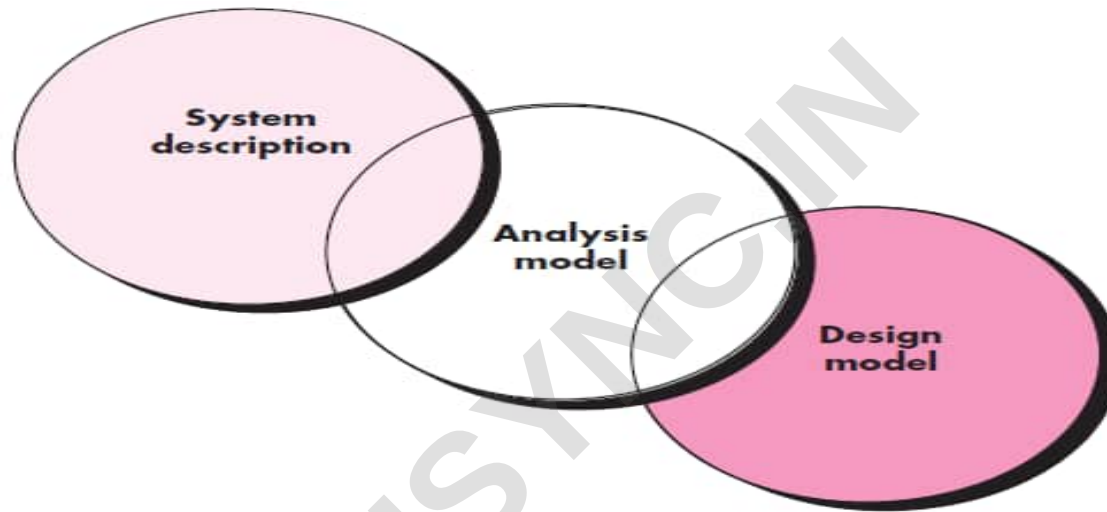


Fig: The requirements model as a bridge between the system description and the design model

6.1.1 Three objectives of requirement model:

- To describe what the customer requires
- To establish a basis for the creation of a software design
- To define a set of requirements that can be validated once the software is built

6.1.2 Analysis Rules of Thumb

Arlow and Neustadt suggest a number of worthwhile rules of thumb that should be followed when creating the analysis model:

- The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high.
- Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function and behavior of the system.
- Delay consideration of infrastructure and other non-functional models until design.
- Minimize coupling throughout the system.
- Be certain that the analysis model provides value to all stakeholders.
- Keep the model as simple as it can be.

6.1.3 Domain Analysis

- Software domain analysis is the identification, analysis, and specification of common requirements from a specific application domain, typically for reuse on multiple projects within that application domain . . . [Object-oriented domain analysis is] the identification, analysis, and specification of common, reusable capabilities within a specific application domain, in terms of common objects, classes, subassemblies, and frameworks . . .

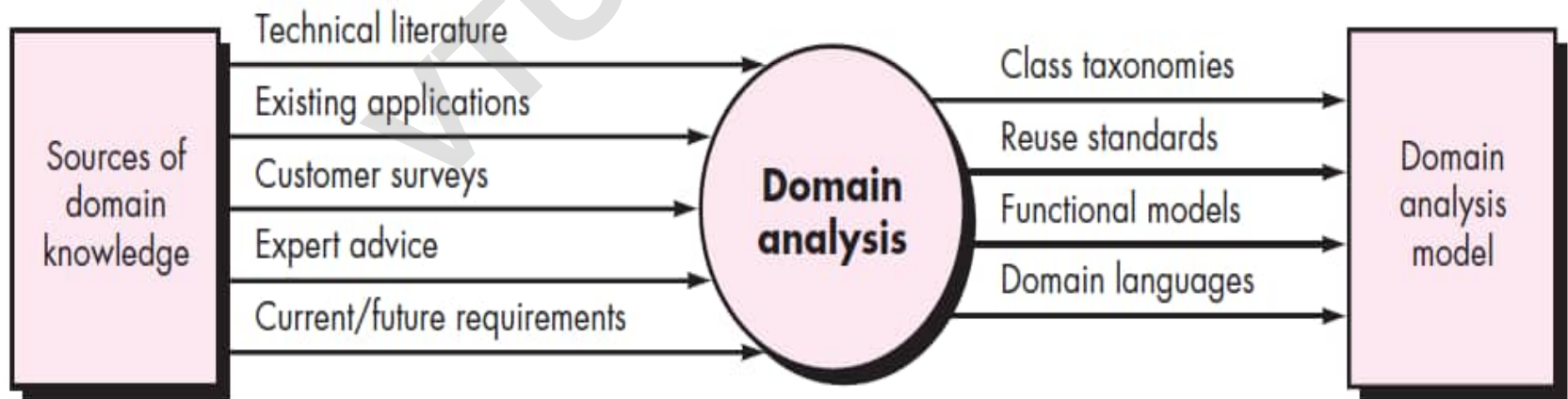
Donald Firesmith

Domain Analysis (Conti..)

- Define the domain to be investigated.
- Collect a representative sample of applications in the domain.
- Analyze each application in the sample.
- Develop an analysis model for the objects.

FIGURE 6.2

Input and output for domain analysis



6.1.4 Requirements Modeling Approaches

- Structural analysis:
 - The **data**: The model defines their *attributes* and *relationships*.
 - The **processes** that *transform* the data: The model shows how they transform the data objects as they *flow* through the system.
- Object-oriented analysis:
 - Focus: **Classes** and their **inter-relationships**
 - UML is predominantly object-oriented

Analysis Model Approaches

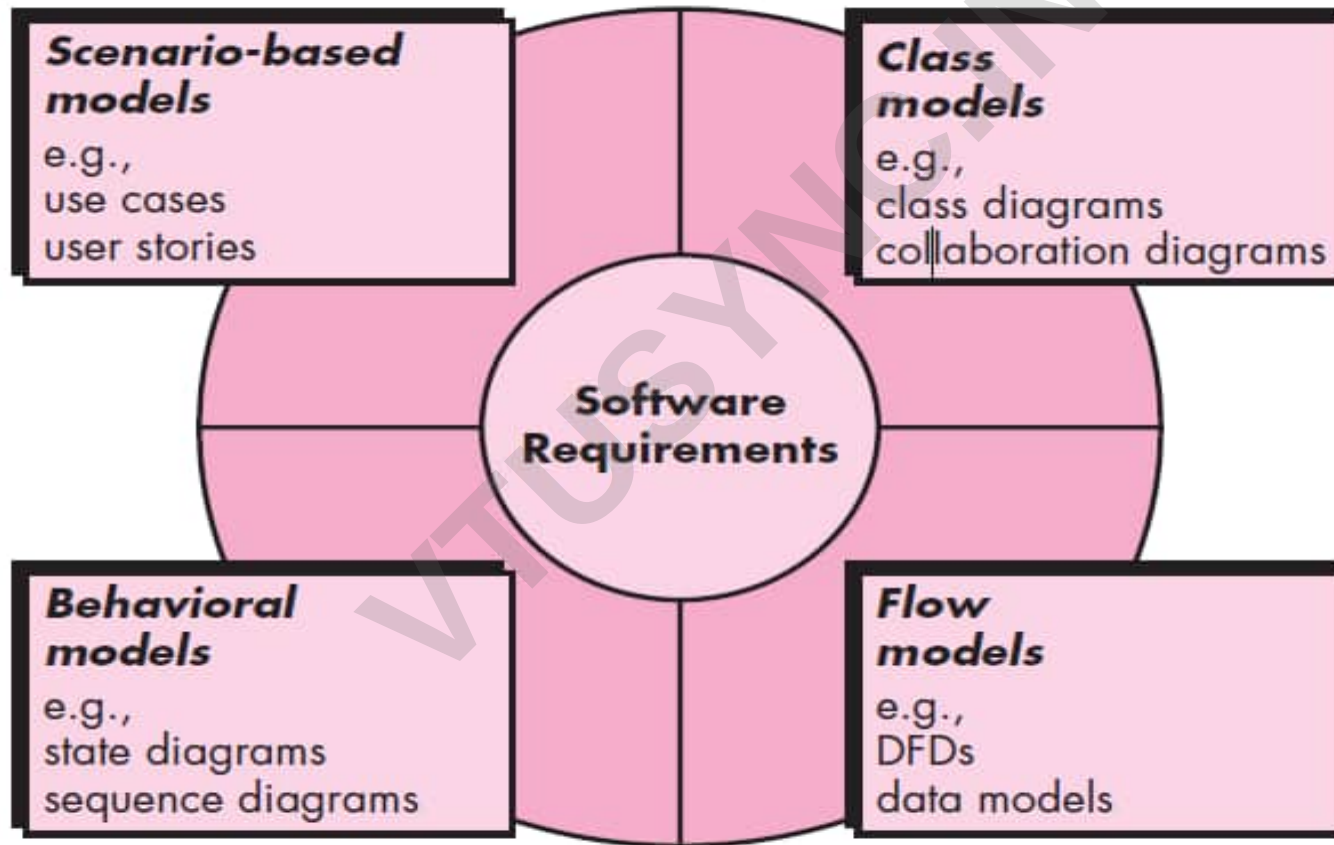
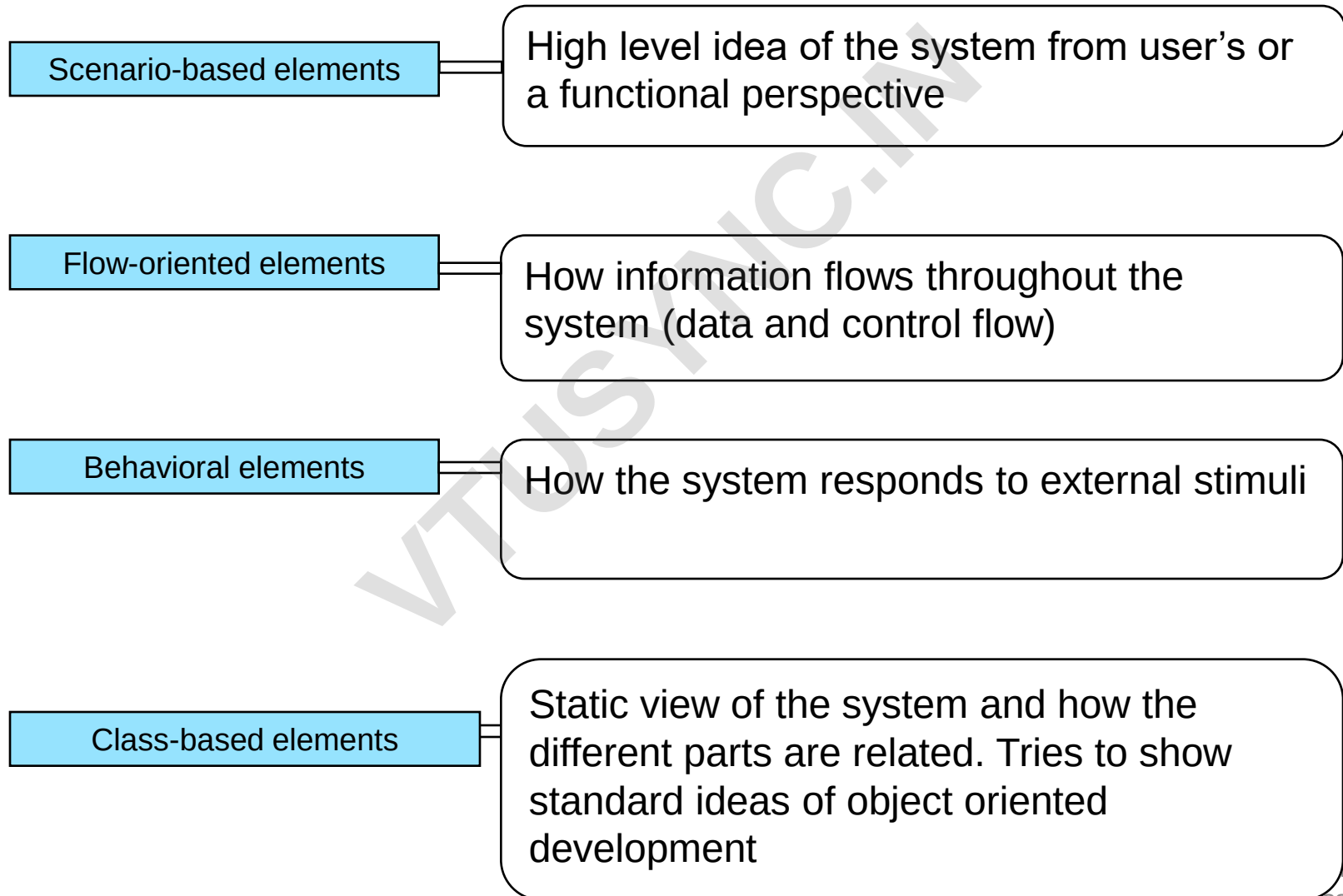


Fig: Elements of the analysis model

Elements of the Analysis Model



6.2 Scenario Based Modeling

- Although the success of a computer-based system or product is measured in many ways, user satisfaction resides at the top of the list.
- If you understand how end users (and other actors) want to interact with a system, software team will be better able to properly characterize requirements and build meaningful analysis and design models.
- Hence, requirements modeling with UML begins with the creation of scenarios in the form of use cases, activity diagrams , swimlane diagrams

I. Creating Use Cases

1. Creating a Preliminary Use case
2. Refining a Preliminary Use case
3. Writing a Formal Use case

6.2.1 Creating a Preliminary Use case

“[Use-cases] are simply an aid to defining what exists outside the system (actors) and what should be performed by the system (use-cases).” Ivar Jacobson

- These are the questions that must be answered if use cases are to provide value as a requirements modeling tool.

(1) What should we write about?

(2) How much should we write about it?

(3) How detailed should we make our description?

(4) How should we organize the description?

What to write about

- Inception and elicitation—provide you with the information you'll need to begin writing use cases.
- Requirements gathering meetings, QFD, and other requirements engineering mechanisms are used to
 1. identify stakeholders
 2. define the scope of the problem
 3. specify overall operational goals
 4. establish priorities
 5. outline all known functional requirements, and
 6. describe the things (objects) that will be manipulated by the system.

To begin developing a set of use cases, list the functions or activities performed by a specific actor.

Creating a Preliminary Use case (Conti..)

- The *SafeHome home surveillance function that are performed by the home owner actor*
 1. Select camera to view.
 2. Request thumbnails from all cameras.
 3. Display camera views in a PC window.
 4. Control pan and zoom for a specific camera.
 5. Selectively record camera output.
 6. Replay camera output.
 7. Access camera surveillance via the Internet.

- **Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)**
- **Actor: homeowner**

If I'm at a remote location, I can use any PC with appropriate browser software to logon to the SafeHome Products website. I enter my user ID and two levels of passwords and once I'm validated, I have access to all functionality for my installed SafeHome system. To access a specific camera view, I select "surveillance" from the major function buttons displayed. I then select "pick a camera" and the floor plan of the house is displayed. I then select the camera that I'm interested in. Alternatively, I can look at thumbnail snapshots from all cameras simultaneously by selecting "all cameras" as my viewing choice. Once I choose a camera, I select "view" and a one-frame-per-second view appears in a viewing window that is identified by the camera ID. If I want to switch cameras, I select "pick a camera" and the original viewing window disappears and the floor plan of the house is displayed again. I then select the camera that I'm interested in. A new viewing window appears.

Creating a Preliminary Use case (Conti..)

- **Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)**
- **Actor: homeowner**
 1. The homeowner logs onto the *SafeHome Products website*.
 2. The homeowner enters his or her user ID.
 3. The homeowner enters two passwords (each at least eight characters in length).
 4. The system displays all major function buttons.
 5. The homeowner selects the “surveillance” from the major function buttons.
 6. The homeowner selects “pick a camera.”
 7. The system displays the floor plan of the house.
 8. The homeowner selects a camera icon from the floor plan.
 9. The homeowner selects the “view” button.
 10. The system displays a viewing window that is identified by the camera ID.
 11. The system displays video output within the viewing window at one frame per second.

6.2.2 Refining a Preliminary Use case

- A description of alternative interactions is essential for a complete understanding of the function that is being described by a use case.
- Therefore, each step in the primary scenario is evaluated by asking the following questions [Sch98a]:
 1. *Can the actor take some other action at this point?*
 2. *Is it possible that the actor will encounter some error condition at this point? If so, what might it be?*
 3. *Is it possible that the actor will encounter some other behavior at this point (e.g., behavior that is invoked by some event outside the actor's control)? If so, what might it be?*

Let us consider

6. The homeowner selects “pick a camera.”
7. The system displays the floor plan of the house.

Use Case Exceptions

- Cockburn [Coc01b] recommends using a “brainstorming” session to derive a reasonably complete set of exceptions .

the following issues should also be explored:

- 1. Are there cases in which some “validation function” occurs during *this use case*?** This implies that validation function is invoked and a potential error condition might occur.
- 2. Are there cases in which a supporting function (or actor) will fail to respond appropriately?** For example, a user action awaits a response but the function that is to respond times out.
- 3. Can poor system performance result in unexpected or improper user actions?** For example, a Web-based interface responds too slowly, resulting in a user making multiple selects on a processing button. These selects queue inappropriately and ultimately generate an error condition.

6.2.3 Writing the Formal Use Case

SafeHome



Use Case Template for Surveillance

Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)

Iteration: 2, last modification: January 14 by V. Raman.

Primary actor: Homeowner.

Goal in context: To view output of camera placed throughout the house from any remote location via the Internet.

Preconditions: System must be fully configured; appropriate user ID and passwords must be obtained.

Trigger: The homeowner decides to take a look inside the house while away.

Scenario:

1. The homeowner logs onto the SafeHome Products website.
2. The homeowner enters his or her user ID.
3. The homeowner enters two passwords (each at least eight characters in length).
4. The system displays all major function buttons.
5. The homeowner selects the "surveillance" from the major function buttons.
6. The homeowner selects "pick a camera."
7. The system displays the floor plan of the house.
8. The homeowner selects a camera icon from the floor plan.
9. The homeowner selects the "view" button.
10. The system displays a viewing window that is identified by the camera ID.
11. The system displays video output within the viewing window at one frame per second.

Exceptions:

1. ID or passwords are incorrect or not recognized—see use case **Validate ID and passwords**.
2. Surveillance function not configured for this system—system displays appropriate error message; see use case **Configure surveillance function**.
3. Homeowner selects "View thumbnail snapshots for all camera"—see use case **View thumbnail snapshots for all cameras**.
4. A floor plan is not available or has not been configured—display appropriate error message and see use case **Configure floor plan**.
5. An alarm condition is encountered—see use case **Alarm condition encountered**.

Priority: Moderate priority, to be implemented after basic functions.

When available: Third increment.

Frequency of use: Moderate frequency.

Channel to actor: Via PC-based browser and Internet connection.

Secondary actors: System administrator, cameras.

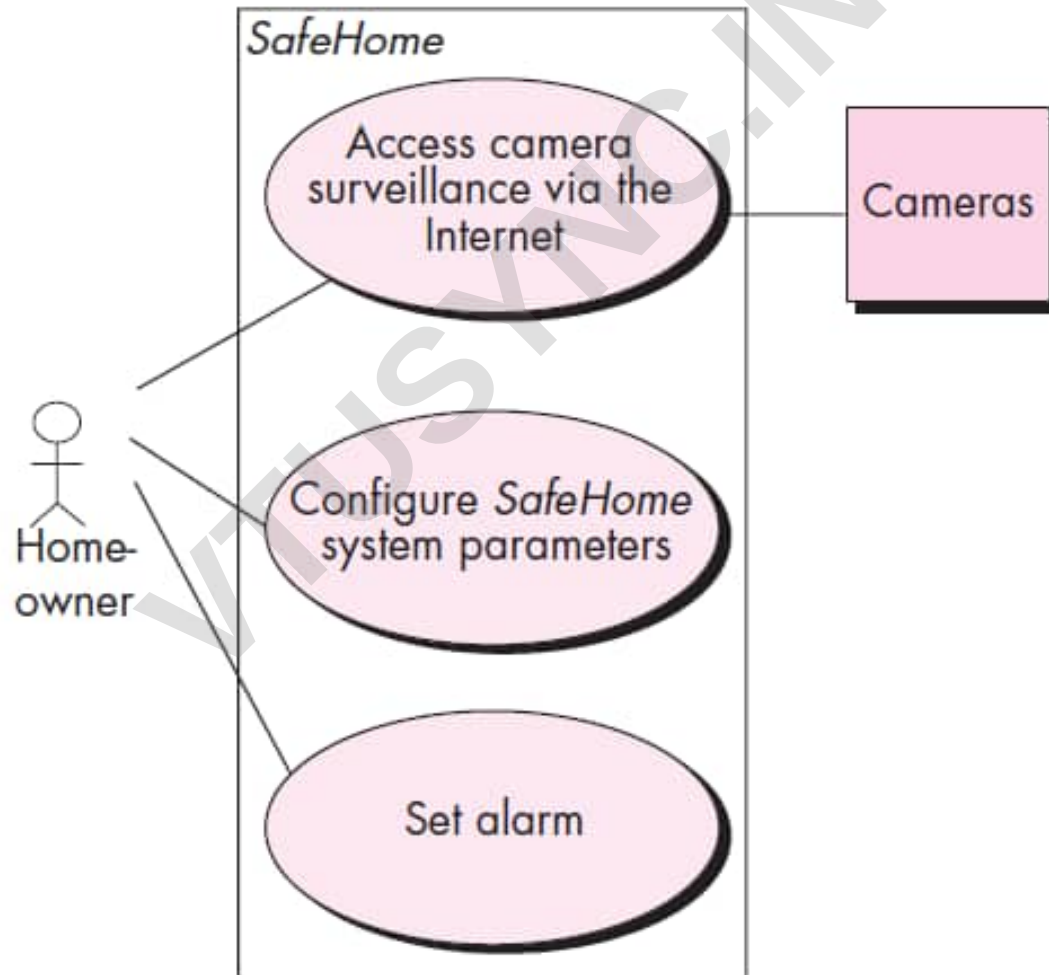
Channels to secondary actors:

1. System administrator: PC-based system.
2. Cameras: wireless connectivity.

Open Issues:

1. What mechanisms protect unauthorized use of this capability by employees of SafeHome Products?
2. Is security sufficient? Hacking into this feature would represent a major invasion of privacy.
3. Will system response via the Internet be acceptable given the bandwidth required for camera views?
4. Will we develop a capability to provide video at a higher frames-per-second rate when high-bandwidth connections are available?

Fig: Preliminary use-case diagram for the *Safe Home* system



Limitations of Use Case

1. a use case is only as good as its author(s). If the description is unclear, the use case can be misleading or ambiguous.
2. A use case focuses on functional and behavioral requirements and is generally inappropriate for nonfunctional requirements.
3. For situations in which the requirements model must have significant detail and precision (e.g., safety critical systems), a use case may not be sufficient.

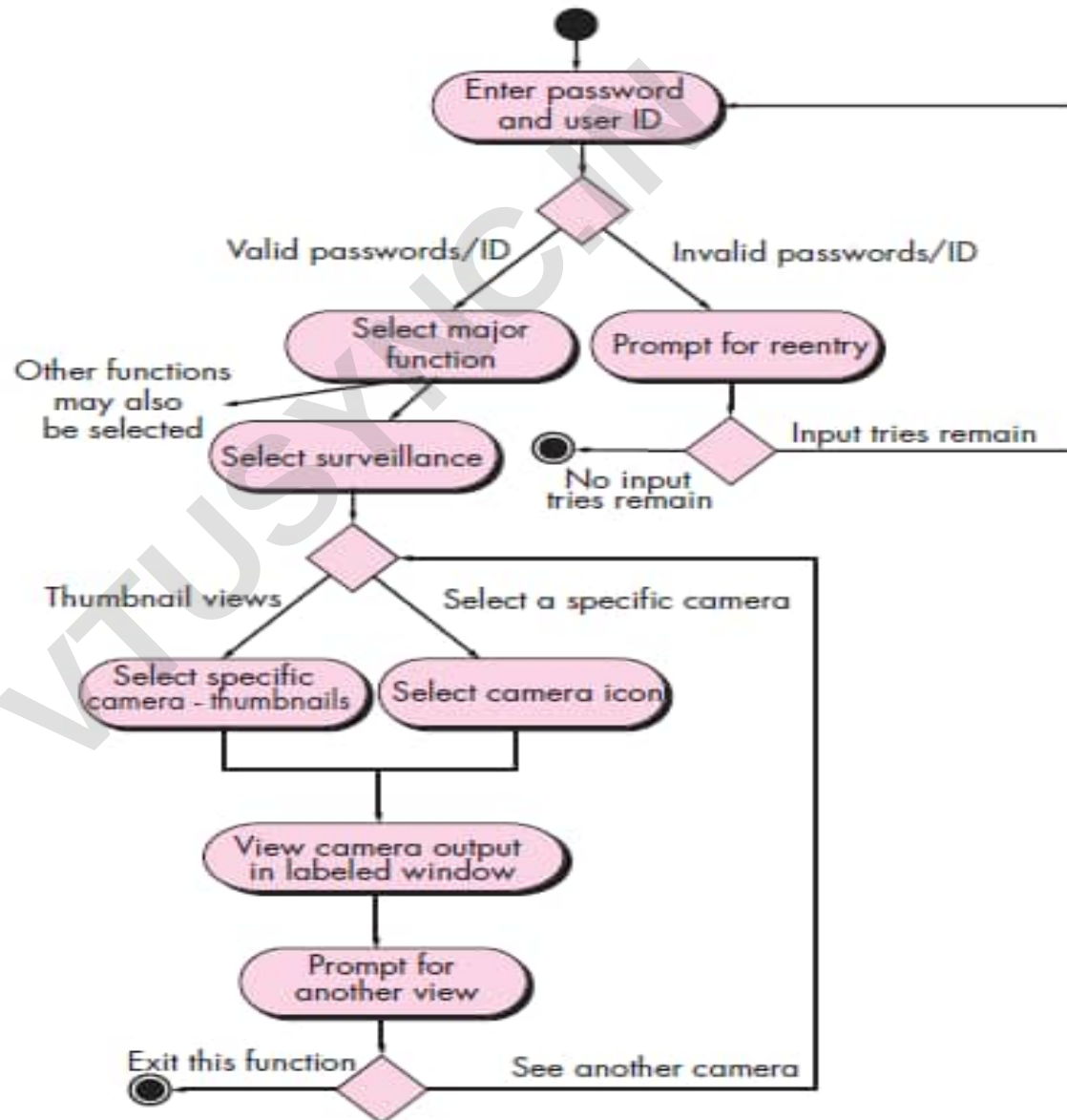
However, scenario-based modeling is appropriate for a significant majority of all situations that you will encounter as a software engineer. If developed properly, the use case can provide substantial benefit as a modeling tool.

6.3 UML MODELS THAT SUPPLEMENT THE USE CASE

II. Developing an Activity Diagram

FIGURE 6.5

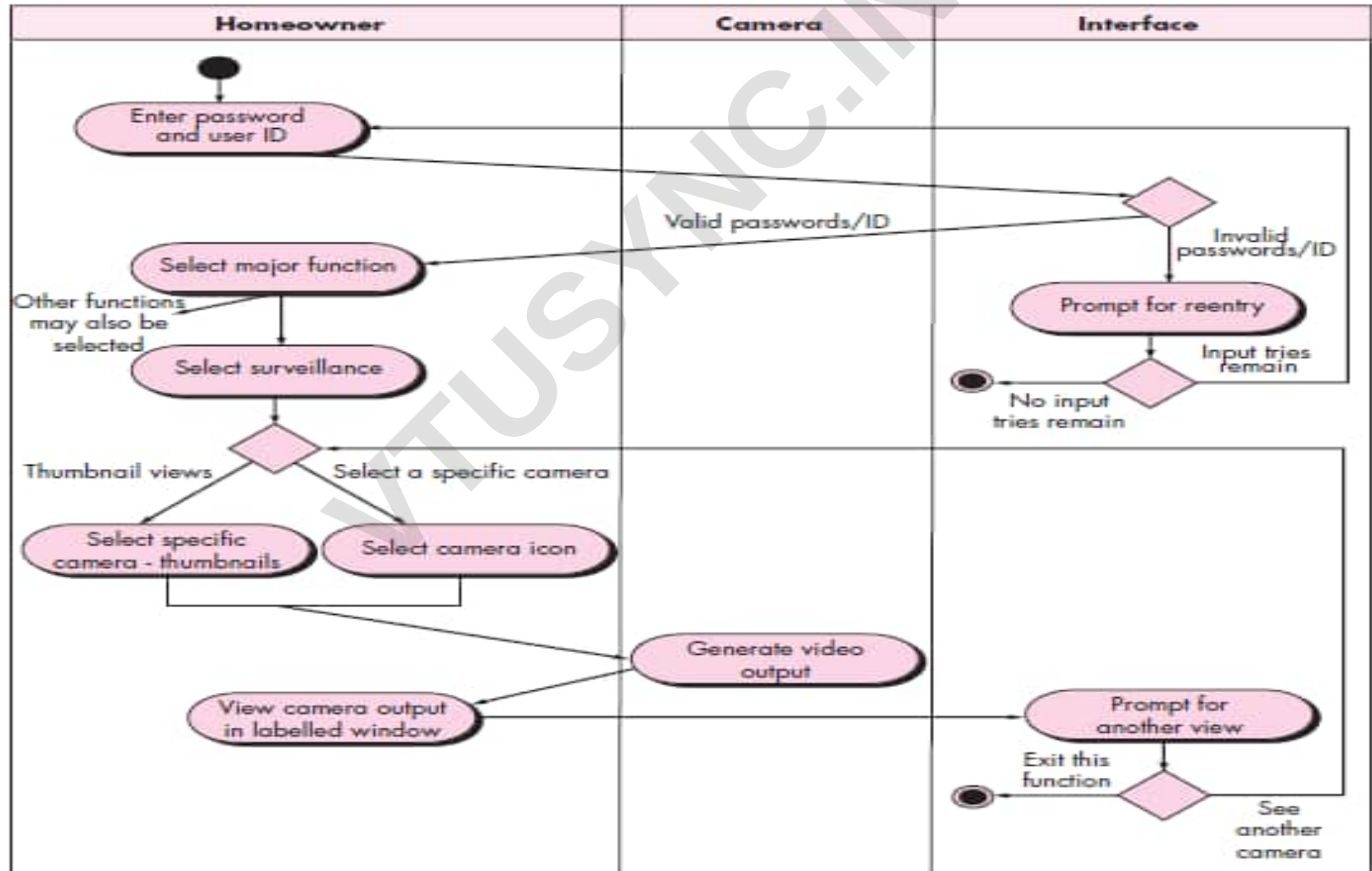
Activity diagram for Access camera surveillance via the Internet—display camera views function.



UML MODELS THAT SUPPLEMENT THE USE CASE (Conti..)

III. Swimlane Diagrams

FIGURE 6.6 Swimlane diagram for Access camera surveillance via the Internet—display camera views function



6.4 Data Modeling Concepts

- If software requirements include the need to create, extend, or interface with a database or if complex data structures must be constructed and manipulated, the software team may choose to create a *data model as part of overall requirements modeling*.
- A software engineer or analyst defines all data objects that are processed within the system, the relationships between the data objects, and other information that is pertinent to the relationships.
- The *entity-relationship diagram (ERD)* addresses these issues and represents all data objects that are entered, stored, transformed, and produced within an application.

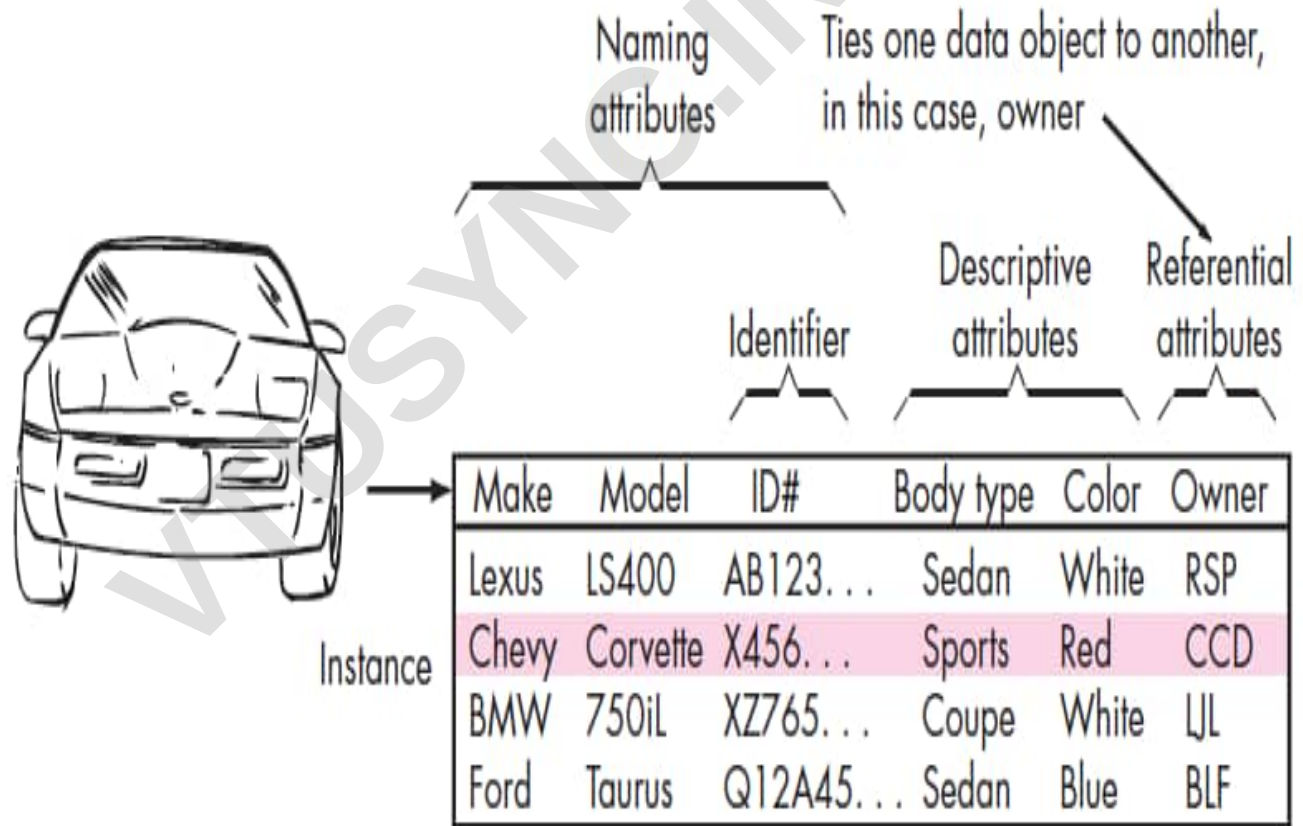
6.4.1 Data Objects

- A data object is a representation of composite information that must be understood by software.
- A data object can be an
 - i. external entity (e.g., anything that produces or consumes information)
 - ii. a thing (e.g., a report or a display)
 - iii. an occurrence (e.g., a telephone call)
 - iv. event (e.g., an alarm)
 - v. a role (e.g., salesperson)
 - vi. an organizational unit (e.g. accounting department), a place (e.g., a warehouse), or a structure (e.g., a file).

Data Objects (Conti..)

FIGURE 6.7

Tabular
representation
of data objects



6.4.2 Data Attributes

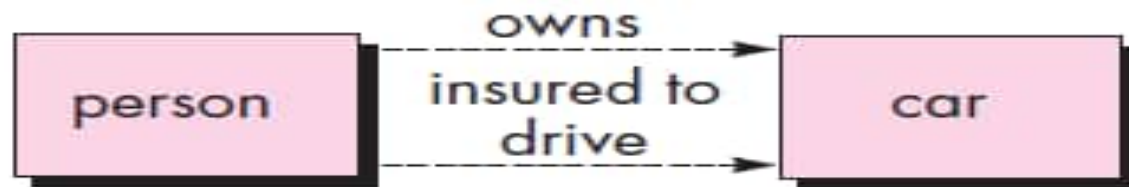
- *Data attributes define the properties of a data object and take on one of three different characteristics.*
- They can be used to
 - (1) name an instance of the data object
 - (2) Describe the instance
 - (3) make reference to another instance in another table
- The set of attributes that is appropriate for a given data object is determined through an understanding of the problem context.
- Eg: (e.g., interior code, drive train type, trim package designator, transmission type) would have to be added to make **car** a meaningful object in the manufacturing control context.

6.4.3 Relationships

- Data objects are connected to one another in different ways.
- Consider the two data objects, **person** and **car**.
- These objects can be represented using the simple notation illustrated in Figure below



(a) A basic connection between data objects



(b) Relationships between data objects

6.5 Class Based modeling

- Class-based modeling represents:
 - **objects** that the system will manipulate
 - **operations** (also called methods or services) that will be applied to the objects to effect the manipulation
 - **relationships** (some hierarchical) between the objects
 - **collaborations** that occur between the classes that are defined.
- The elements of a class-based model include classes and objects, attributes, operations, class responsibility- collaborator (CRC) models, collaboration diagrams, and packages.

6.5.1 Identifying Analysis Classes

- Examining the usage scenarios developed as part of the requirements model and perform a "grammatical parse" [Abb83]
- Classes are determined by underlining each noun or noun phrase and entering it into a simple table.
- Synonyms should be noted.
- If the class (noun) is required to implement a solution, then it is part of the solution space; otherwise, if a class is necessary only to describe a solution, it is part of the problem space.
- But what should we look for once all of the nouns have been isolated?

Manifestations of Analysis Classes

Analysis classes manifest themselves in one of the following ways:

1. ***External entities*** (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
2. ***Things*** (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
3. ***Occurrences or events*** (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
4. ***Roles*** (e.g., manager, engineer, salesperson) played by people who interact with the system.
5. ***Organizational units*** (e.g., division, group, team) that are relevant to an application.
6. ***Places*** (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
7. ***Structures*** (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

The SafeHome security function *enables* the homeowner to *configure* the security system when it is *installed*, *monitors* all sensors *connected* to the security system, and *interacts* with the homeowner through the Internet, a PC, or a control panel.

During installation, the SafeHome PC is used to *program* and *configure* the system. Each sensor is assigned a number and type, a master password is programmed for *arming* and *disarming* the system, and telephone number(s) are *input* for *dialing* when a sensor event occurs.

When a sensor event is *recognized*, the software *invokes* an audible alarm attached to the system. After a delay time that is *specified* by the homeowner during system configuration activities, the software dials a telephone number of a monitoring service, *provides information* about the location, *reporting* the nature of the event that has been detected. The telephone number will be *redialed* every 20 seconds until telephone connection is *obtained*.

The homeowner *receives* security information via a control panel, the PC, or a browser, collectively called an interface. The interface *displays* prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form . . .

Potential Class

homeowner

sensor

control panel

installation

system (alias security system)

number, type

master password

telephone number

sensor event

audible alarm

monitoring service

General Classification

role or external entity

external entity

external entity

occurrence

thing

not objects, attributes of sensor

thing

thing

occurrence

external entity

organizational unit or external entity

Six Selection Characteristics

- 1. *Retained information.*** Does the system need to remember data about this class?
- 2. *Needed services.*** Does the class provide identifiable operations or behaviors?
- 3. *Multiple attributes.*** Does the class have multiple important attributes?
- 4. *Common attributes.*** Do the attributes of the potential class apply to all instances of the class?
- 5. *Common operations.*** Are there operations that apply to all instances of the class?
- 6. *Essential requirements.*** Is the class essential for producing or consuming information in the system?

Potential Class	Characteristic Number That Applies
X homeowner	rejected: 1, 2 fail even though 6 applies
sensor	accepted: all apply
control panel	accepted: all apply
X installation	rejected
system (alias security function)	accepted: all apply
X number, type	rejected: 3 fails, attributes of sensor
X master password	rejected: 3 fails
X telephone number	rejected: 3 fails
sensor event	accepted: all apply
audible alarm	accepted: 2, 3, 4, 5, 6 apply
X monitoring service	rejected: 1, 2 fail even though 6 applies

It should be noted that (1) the preceding list is not all-inclusive, additional classes would have to be added to complete the model; (2) some of the rejected potential classes will become attributes for those classes that were accepted (e.g., **number** and **type** are attributes of **Sensor**, and **master password** and **telephone number** may become attributes of **System**); (3) different statements of the problem might cause different “accept or reject” decisions to be made (e.g., if each homeowner had an individual password or was identified by voice print, the **Homeowner** class would satisfy characteristics 1 and 2 and would have been accepted).

6.5.2 Defining Attributes

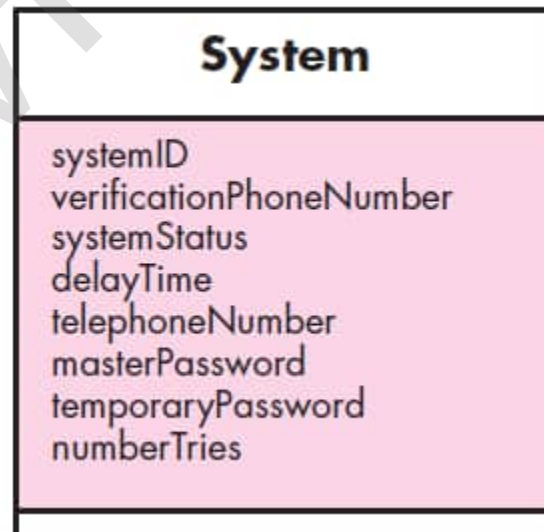
- *Attributes* describe a class that has been selected for inclusion in the analysis model.
- build two different classes for professional baseball players
 - **For Players Statistics software:** name, position, batting average, fielding percentage, years played, and games played might be relevant
 - **For Pension Fund software:** average salary, credit toward full vesting, pension plan options chosen, mailing address, and the like.

- To illustrate, we consider the **System** class defined for *SafeHome*. A *homeowner* can configure the security function to reflect **sensor information, alarm response information, activation/deactivation information, identification information,**

identification information = system ID + verification phone number + system status

alarm response information = delay time + telephone number

activation/deactivation information = master password + number of allowable tries + temporary password



6.5.3 Defining Operations

- Operations define the behavior of an object.
- Operations can be divided into four broad categories:
 - (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting)
 - (2) operations that perform a computation
 - (3) operations that inquire about the state of an object
 - (4) operations that monitor an object for the occurrence of a controlling event.
- Do a grammatical parse of a processing narrative and look at the verbs

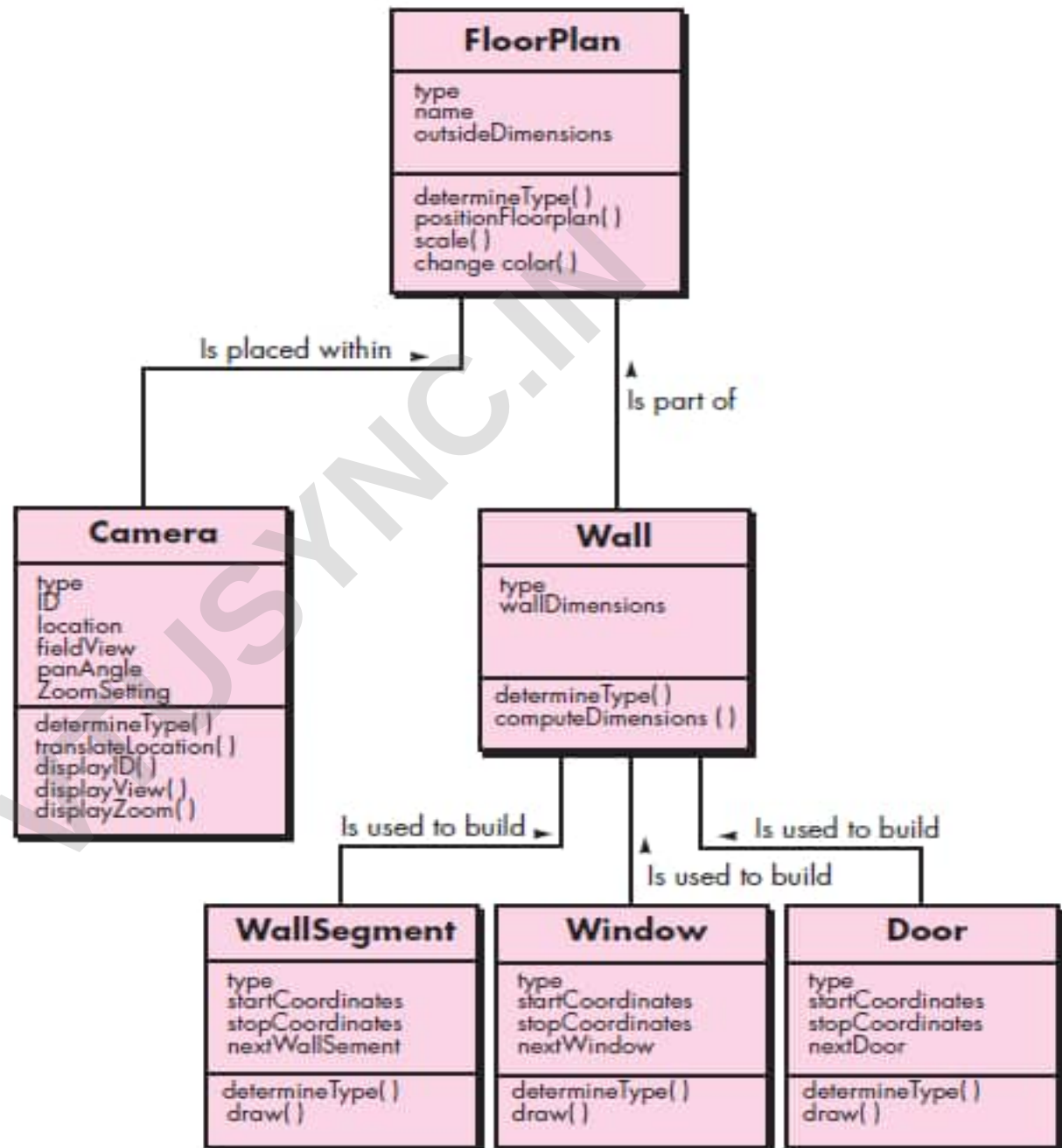
Defining Operations (Conti..)

- For example, from the *SafeHome* processing narrative presented earlier in this chapter, we see that “**sensor is assigned a number and type**” or “**a master password is programmed for arming and disarming the system.**” These phrases indicate a number of things:
 1. That an *assign()* operation is relevant for the **Sensor class**.
 2. That a *program()* operation will be applied to the **System class**.
 3. That *arm()* and *disarm()* are operations that apply to **System class**.

System
systemID verificationPhoneNumber systemStatus delayTime telephoneNumber masterPassword temporaryPassword numberTries
program() display() reset() query() arm() disarm()

FIGURE 6.10

Class diagram
for FloorPlan
(see sidebar
discussion)



6.5.4 Class – Responsibility Collaborator (CRC) Modeling

- *Class-responsibility-collaborator (CRC) modeling [Wir90] provides a simple means for identifying and organizing the classes that are relevant to system or product requirements.*
- A CRC model is really a collection of **standard index cards that represent classes**. The cards are divided into three sections. Along the top of the card you write the name of the class. In the body of the card you list the class responsibilities on the left and the collaborators on the right.
- *Responsibilities are the attributes and operations that are relevant for the class. Stated simply, a responsibility is “anything the class knows or does” [Amb95].*
- *Collaborators are those classes that are required to provide a class with the information needed to complete a responsibility. a collaboration implies either a request for information or a request for some action.*

FIGURE 6.11

A CRC model
index card

Class: FloorPlan	
Description	
Responsibility:	Collaborator:
Defines floor plan name/type	
Manages floor plan positioning	
Scales floor plan for display	
Scales floor plan for display	
Incorporates walls, doors, and windows	Wall
Shows position of video cameras	Camera

Classes

1. Entity classes(model or business classes): typically represent things that are to be stored in a database and persist throughout the duration of the application (e.g., **FloorPlan and Sensor**).
2. Boundary classes : *are used to create the interface (e.g., interactive screen or printed reports) that the user sees and interacts with as the software is used. Boundary classes are designed with the responsibility of managing the way entity objects are represented to users(Eg: **CameraWindow**)*
3. Controller classes: *manage a “unit of work” from start to finish. That is, controller classes can be designed to manage*
 1. the creation or update of entity objects,
 2. the instantiation of boundary objects as they obtain information from entity objects,
 3. complex communication between sets of objects
 4. validation of data communicated between objects or between the user and the application.

In general, controller classes are not considered until the design activity has begun.

Responsibilities.

- **Wirfs-Brock and her colleagues [Wir90] suggest five guidelines for allocating responsibilities to classes:**
 1. System intelligence should be distributed across classes to best address the needs of the problem
 2. Each responsibility should be stated as generally as possible
 3. Information and the behavior related to it should reside within the same class
 4. Information about one thing should be localized with a single class, not distributed across multiple classes.
 5. Responsibilities should be shared among related classes, when appropriate.
- Eg: As an example, consider a video game that must display the following classes: **Player, PlayerBody, PlayerArms, PlayerLegs, PlayerHead**. Each of these classes has its own attributes (e.g., position, orientation, color, speed) and all must be updated and displayed as the user manipulates a joystick.

Collaborations

■ Classes fulfill their responsibilities in one of two ways:

1. A class can use its own operations to manipulate its own attributes, thereby fulfilling a particular responsibility, or
2. A class can collaborate with other classes.

- Wirfs-Brock and her colleagues [Wir90] define collaborations in the following way:

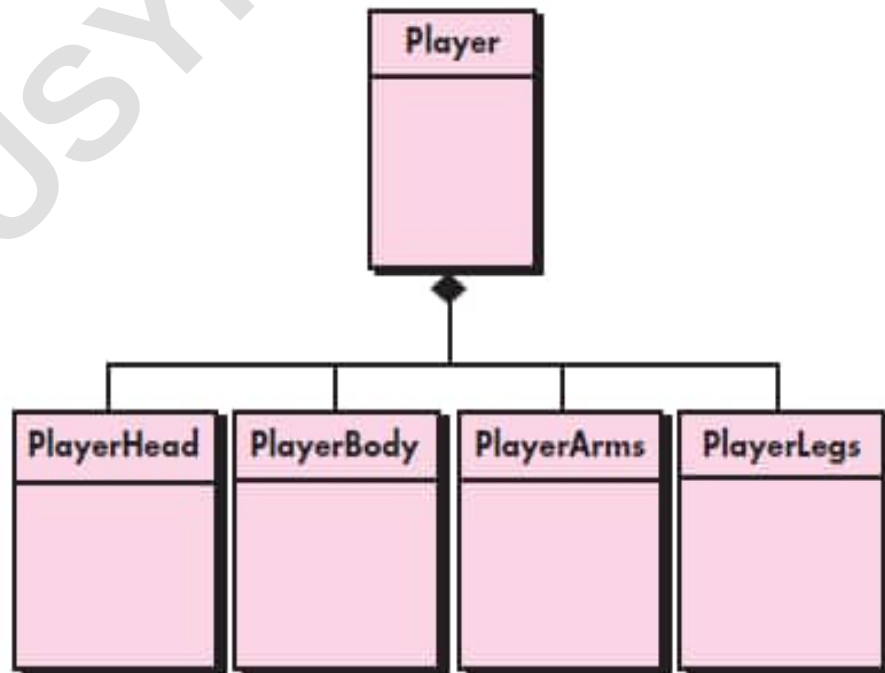
“

Collaborations represent requests from a client to a server in fulfillment of a client responsibility. A collaboration is the embodiment of the contract between the client and the server. We say that an object collaborates with another object if, to fulfill a responsibility, it needs to send the other object any messages. A single collaboration flows in one direction—representing a request from the client to the server. From the client's point of view, each of its collaborations is associated with a particular responsibility implemented by the server”

- Eg: Security function of safe home system
- To help in the identification of collaborators, you can examine three different generic relationships between classes
 1. the *is-part-of* relationship
 2. the *has-knowledge-of* relationship
 3. the *depends-upon* relationship

FIGURE 6.12

A composite
aggregate
class



- When a complete CRC model has been developed, stakeholders can review the model using the following approach [Amb95]:
 1. **All participants in the review (of the CRC model) are given a subset of the CRC model index cards.** Cards that collaborate should be separated (i.e., no reviewer should have two cards that collaborate).
 2. **All use-case scenarios (and corresponding use-case diagrams) should be organized into categories.**
 3. **The review leader reads the use case deliberately.** As the review leader comes to a named object, she passes a token to the person holding the corresponding class index card. For example, a use case for *SafeHome* contains the following narrative:

“The homeowner observes the SafeHome control panel to determine if the system is ready for input. If the system is not ready, the homeowner must physically close windows/doors so that the ready indicator is present. [A not-ready indicator implies that a sensor is open, i.e., that a door or window is open.]”

The phrase “implies that a sensor is open” requires that the index card contains a responsibility that will validate this implication (the responsibility *determinesensor- status()* *accomplishes this*). Next to the responsibility on the index card is the collaborator **Sensor**. The token is then passed to the **Sensor object**.

4. **When the token is passed, the holder of the Sensor card is asked to describe** the responsibilities noted on the card. The group determines whether one (or more) of the responsibilities satisfies the use-case requirement.
5. **If the responsibilities and collaborations noted on the index cards cannot** accommodate the use case, modifications are made to the cards.

This modus operandi continues until the use case is finished. When all use cases have been reviewed, requirements modeling continues.

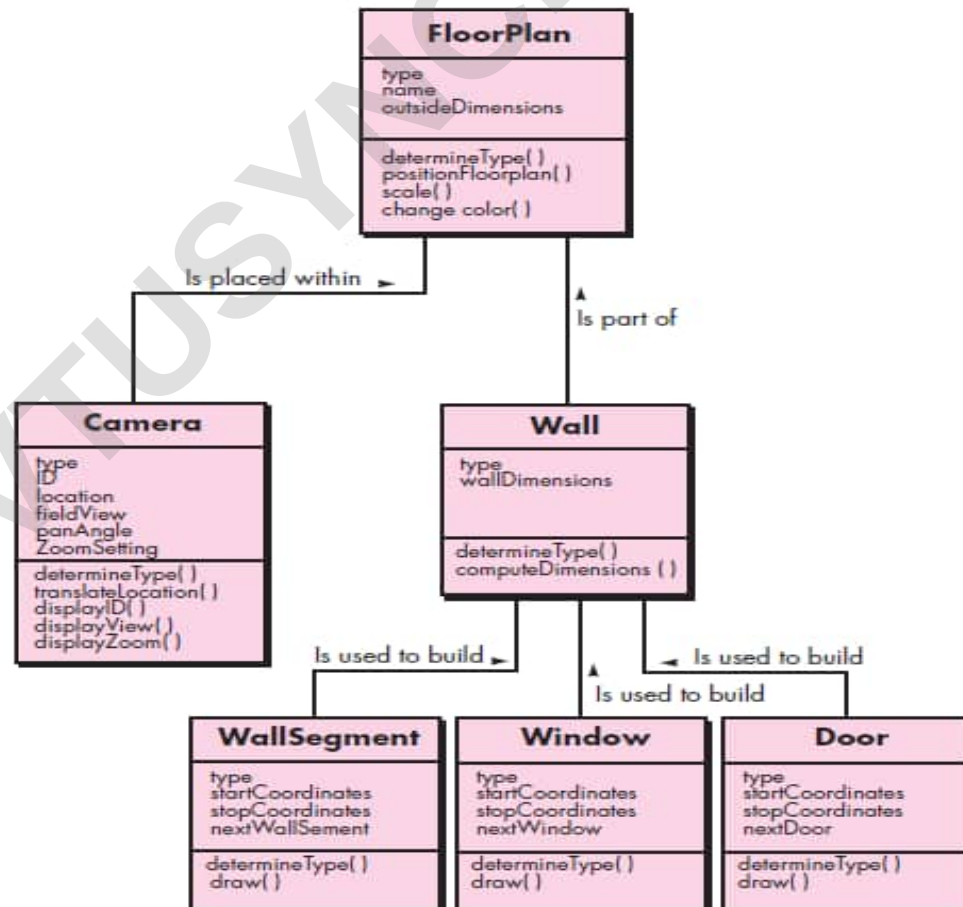
6.5.5 Associations and Dependencies

1. Associations:

- Two analysis classes are often related to one another in some fashion
 - In UML these relationships are called *associations*

FIGURE 6.10

Class diagram
for FloorPlan
(see sidebar
discussion)



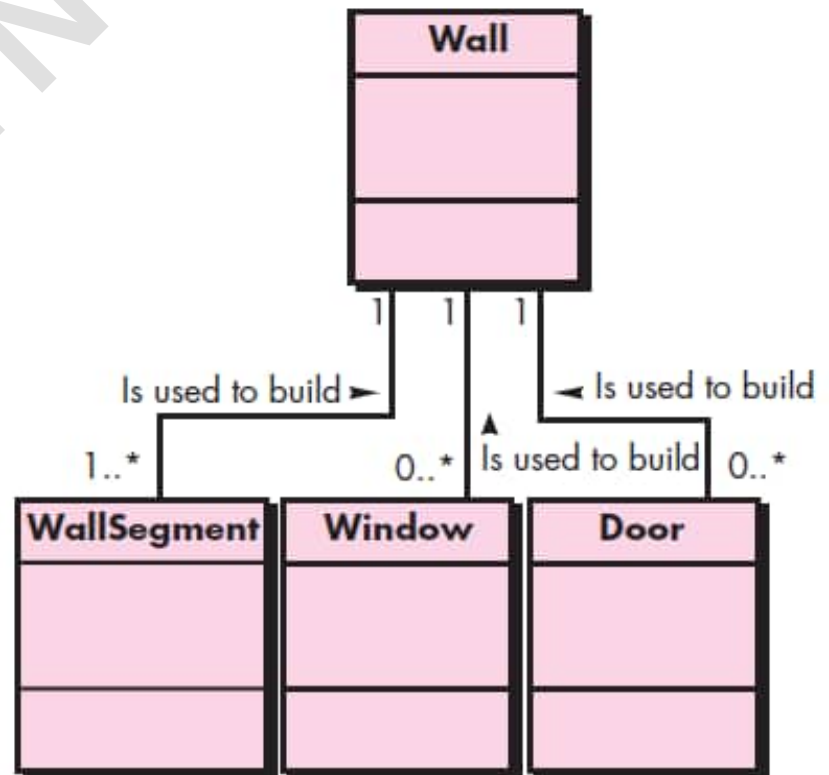
6.5.5 Associations and Dependencies

2. Multiplicity: Associations can be refined by indicating *multiplicity*

- where “one or more” is represented using 1..*, and “0 or more” by 0..*. In UML, the
- asterisk indicates an unlimited upper bound on the range

FIGURE 6.13

Multiplicity

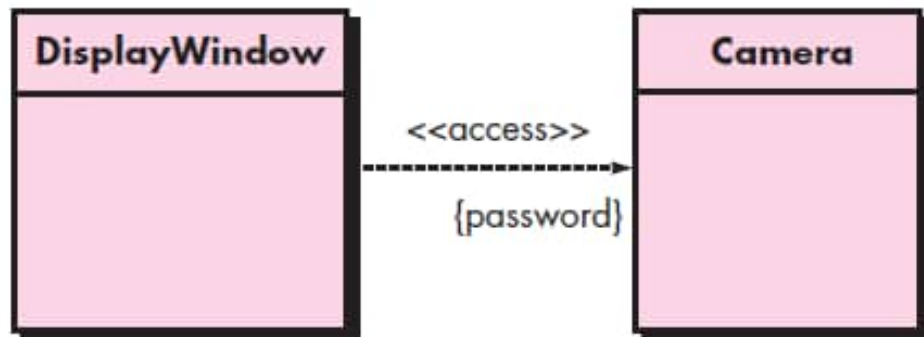


6.5.5 Associations and Dependencies

- 3. Dependency:** In many instances, a client-server relationship exists between two analysis classes.
- In such cases, a client class depends on the server class in some way and a *dependency relationship is established*.
 - *Dependencies are defined by a stereotype. A stereotype is an “extensibility mechanism”*
 - In UML stereotypes are represented in double angle brackets (e.g., <<stereotype>>).
 - As shown in Figure 6.14 where <<access>> implies that the use of the camera output is controlled by a special password.

FIGURE 6.14

Dependencies

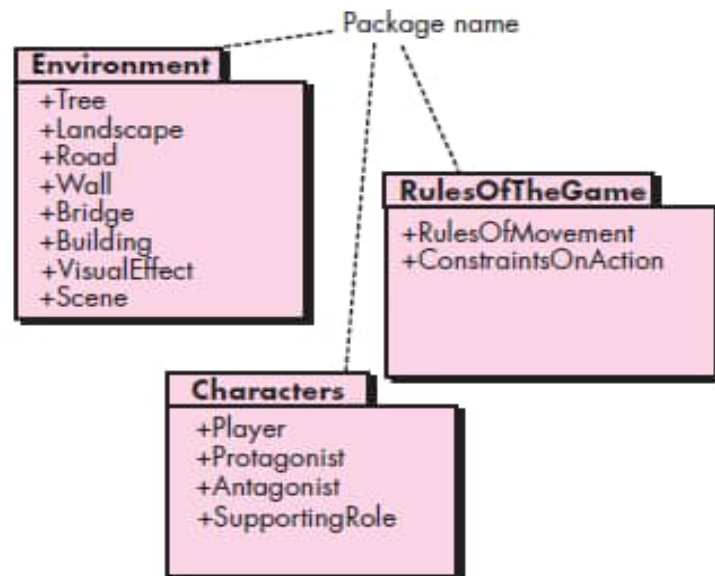


6.5.6 Analysis Packages

- An important part of analysis modeling is categorization.
- That is, various elements of the analysis model (e.g., use cases, analysis classes) are categorized in a manner that packages them as a grouping—called an *analysis package*—that is given a representative name.
- + sign in the analysis package indicates the classes have public visibility and are therefore accessible from other packages.
- - sign indicates that an element is hidden from all other packages
- # symbol indicates that an element is accessible only to packages contained within a given package.

FIGURE 6.15

Packages



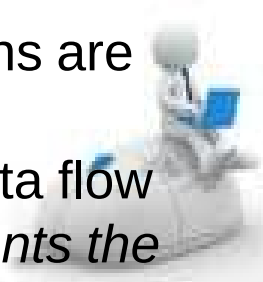
Requirements Modeling: Flow , Behavior Oriented Modeling

Flow-Oriented Modeling 7.2

- Represents how data objects are transformed as they move through the system
- can be used to complement UML diagrams and provide additional insight into system requirements and flow.

○ **data flow diagram (DFD)** is the diagrammatic form that is used

- Data objects are represented by labeled arrows, and transformations are represented by circles (also called bubbles).
- The DFD is presented in a hierarchical fashion. That is, the first data flow model (sometimes called a level 0 DFD or *context diagram*) represents the system as a whole.

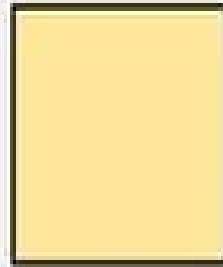


The Flow Model

Every computer-based system is an information transform



Flow Modeling Notation



external entity



process



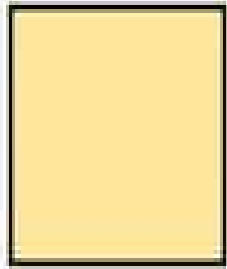
data flow



data store



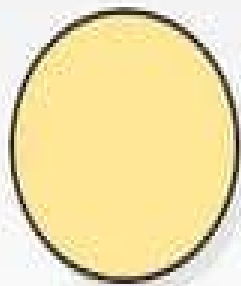
External Entity



- **A producer or consumer of data**
- *Examples: a person, a device, a sensor*
- *Another example: computer-based system*
- *Data must always originate some where and must always be sent to something*



Process

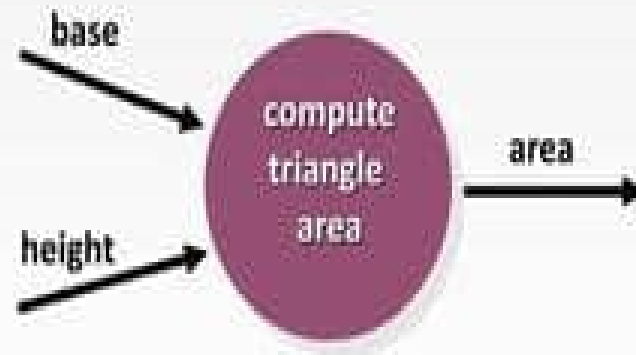
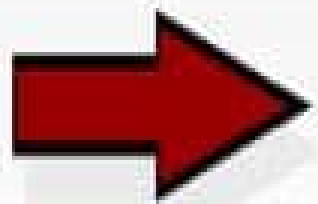


- A data transformer (changes input to output)
- Examples: *compute taxes, determine area,*
- *format report, display graph*
- *Data must always be processed in some way to achieve system function*



Data Flow

- Data flows through a system, beginning as input and transformed into output.

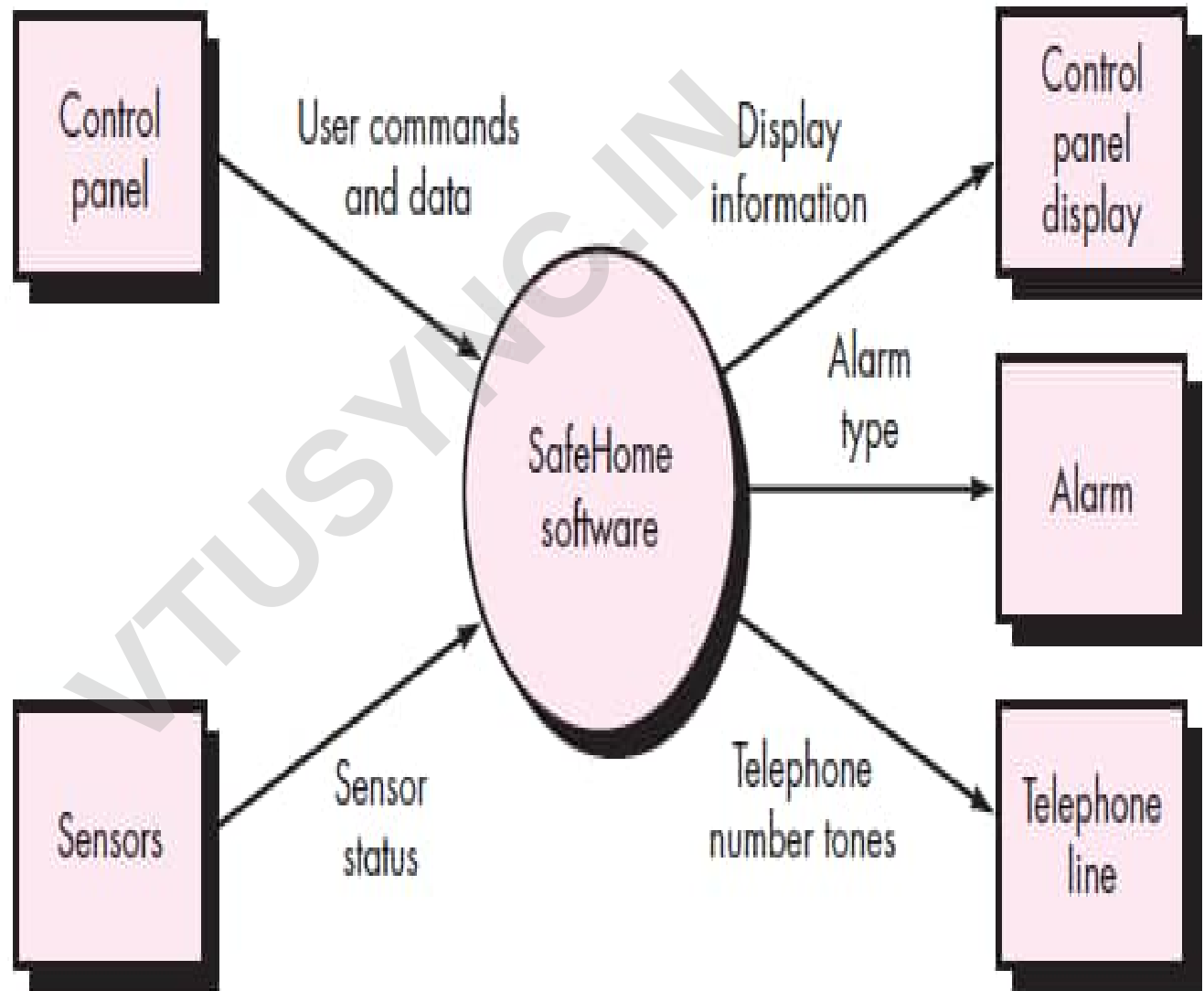


7.2.1 Creating a Data Flow Model

- The data flow diagram enables you to develop models of the information domain and functional domain.
- As the DFD is refined into greater levels of detail, you perform an implicit functional decomposition of the system. At the same time, the DFD refinement results in a corresponding refinement of data as it moves through the processes that embody the application.
- A few simple guidelines can aid immeasurably during the derivation of a data flow diagram:
 1. the level 0 data flow diagram should depict the software/system as a single bubble;
 2. primary input and output should be carefully noted;
 3. Refinement should begin by isolating candidate processes, data objects, and data stores to be represented at the next level
 4. all arrows and bubbles should be labeled with meaningful names;
 5. *information flow continuity must be maintained from level to level,*
 6. one bubble at a time should be refined.

FIGURE 7.1

Context-level
DFD for the
SafeHome
security
function



Conti..

- *The level 0 DFD must now be expanded into a level 1 data flow model, by applying a “grammatical parse” [Abb83] to the use case narrative that describes the context-level bubble. That is, we isolate all nouns (and noun phrases) and verbs (and verb phrases)*

The SafeHome security function enables the homeowner to configure the security system when it is *installed*, *monitors* all sensors *connected* to the security system, and *interacts* with the homeowner through the Internet, a PC, or a control panel.

During installation, the *SafeHome* PC is used to *program* and *configure* the system. Each sensor is assigned a number and type, a master password is programmed for *arming* and *disarming* the system, and telephone number(s) are *input* for *dialing* when a sensor event occurs.

When a sensor event is *recognized*, the software *invokes* an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software dials a telephone number of a monitoring service, *provides information* about the location, *reporting* the nature of the event that has been detected. The telephone number will be *redialed* every 20 seconds until telephone connection is *obtained*.

The homeowner *receives* security information via a control panel, the PC, or a browser, collectively called an interface. The interface *displays* prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form . . .

FIGURE 7.2

Level 1 DFD for
SafeHome
security function

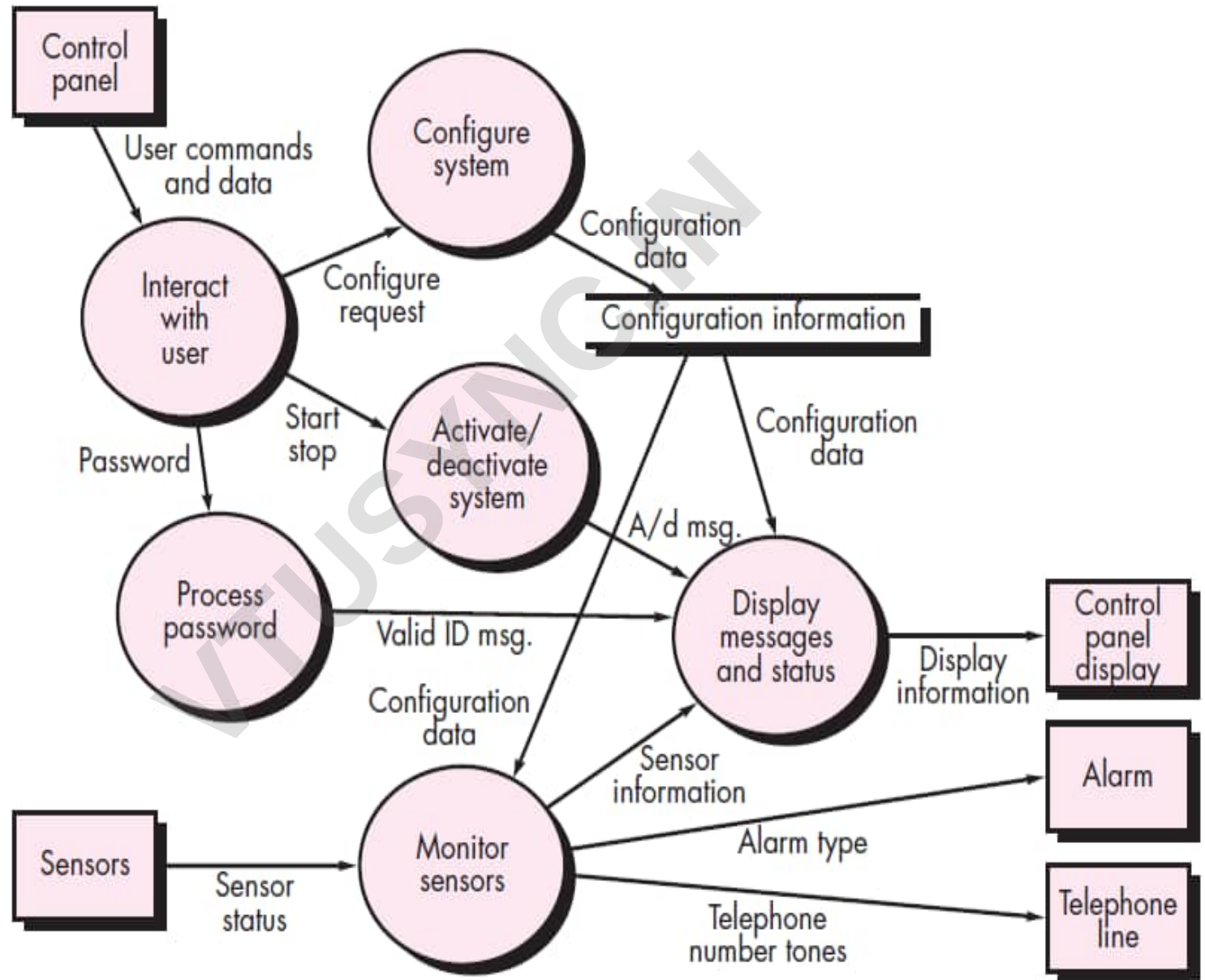
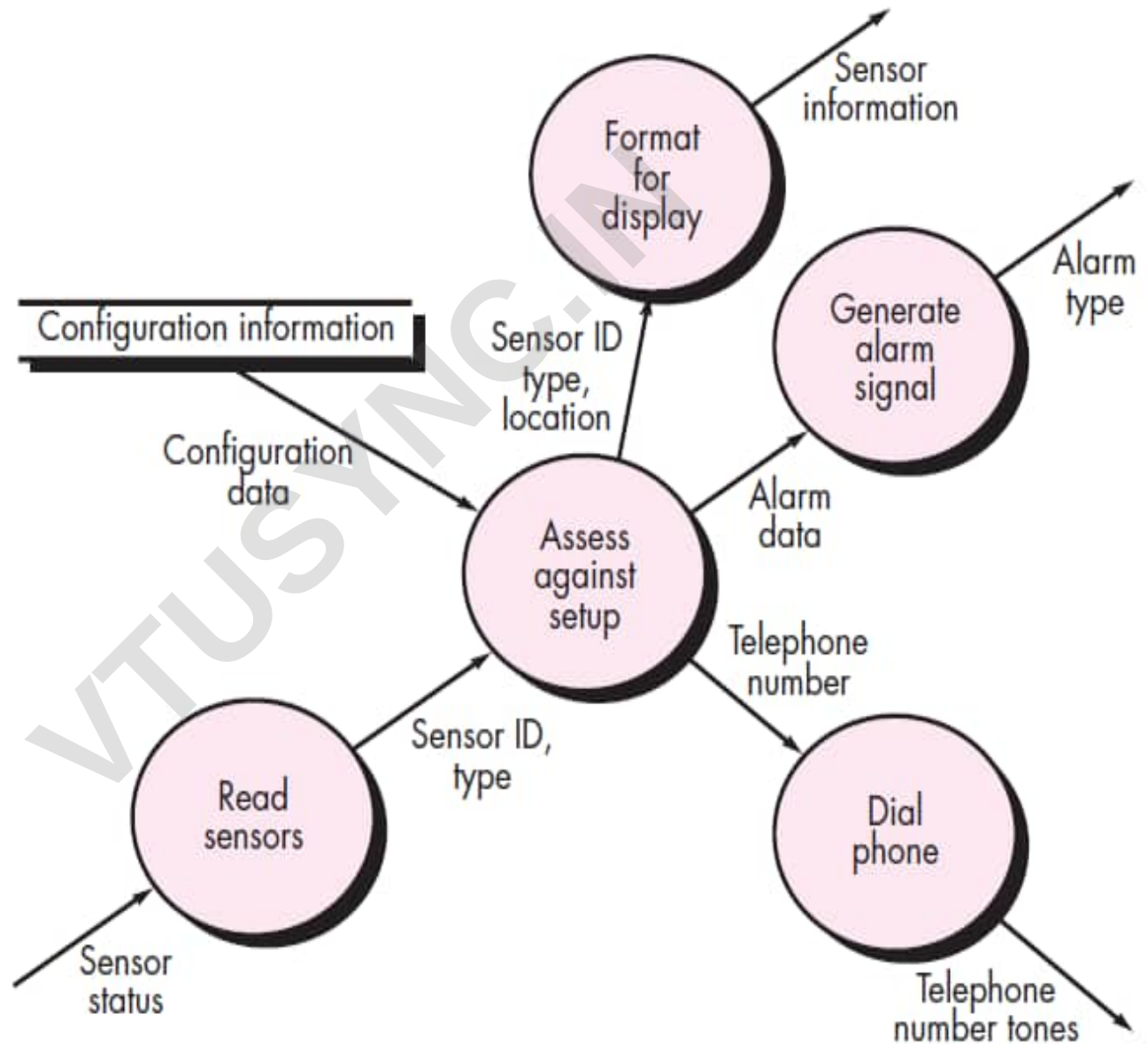


FIGURE 7.3

Level 2 DFD
that refines
the *monitor
sensors* process



Creating a Control Flow Model 7.2.2

- For some types of applications, the data model and the data flow diagram are all that is necessary to obtain meaningful insight into software requirements.
- large class of applications are “driven” by events rather than data
- produce control information rather than reports or displays, and process information with heavy concern for time and performance
- Such applications require the use of control flow modeling in addition to data flow modeling.



Creating a Control Flow Model .cont

□ event or control item

○ To select potential candidate events, the following guidelines are suggested:

- List all sensors that are “read” by the software.
- List all interrupt conditions.
- List all “switches” that are actuated by an operator.
- List all data conditions.
- Recalling the noun/verb parse that was applied to the processing narrative,
 - review all “control items” as possible control specification inputs/outputs.
- Describe the behavior of a system by identifying its states, identify how each
 - state is reached, and define the transitions between states.
- Focus on possible omissions—a very common error in specifying control; for
 - example, ask: “Is there any other way I can get to this state or exit from it?”

Among the many events and control items that are part of *Safe Home software* are

1. **sensor event** (i.e., a sensor has been tripped)
2. **blink flag** (a signal to blink the display)
3. **start/stop switch** (a signal to turn the system on or off).



Control Specification 7.2.3

□ A control specification (CSPEC) represents the behavior of the system in two ways

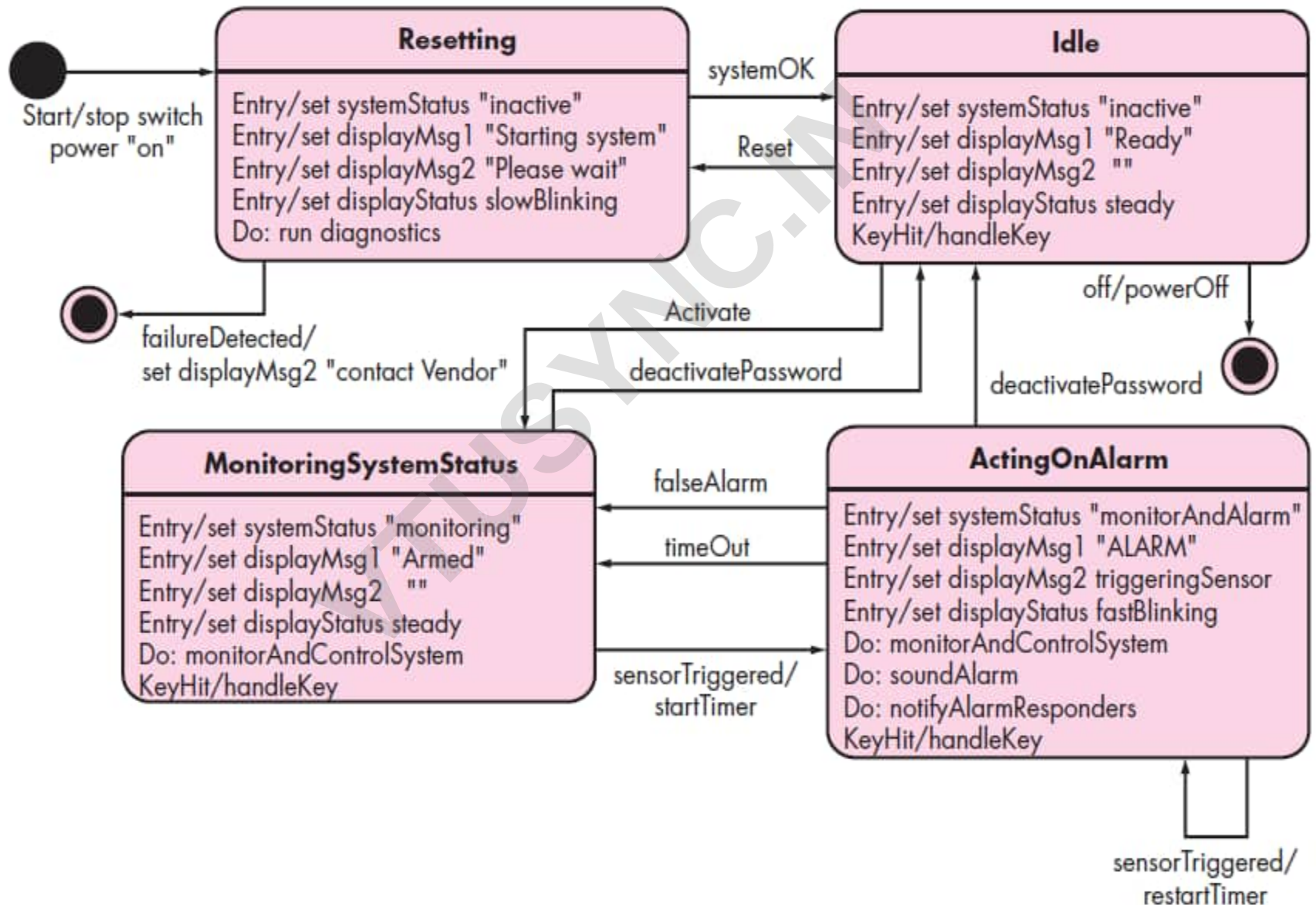
- The CSPEC contains a state diagram that is a sequential specification of behavior.
- It can also contain a program activation table—a combinatorial specification of behavior

○ CSPEC can be:

- state diagram
- state transition table
- decision tables
- activation tables

for example:
Safe home



FIGURE 7.4State diagram for *SafeHome* security function

PAT

- A somewhat different mode of behavioral representation is the process activation table
- The PAT represents information contained in the state diagram in the context of processes, not states
- the table indicates which processes (bubbles) in the flow model will be invoked when an event occurs
- can be used as a guide for a designer who must build an executive that controls the processes represented at this level



FIGURE 7.5

Process activation table for *SafeHome* security function

<u>input events</u>						
sensor event	0	0	0	0	1	0
blink flag	0	0	1	1	0	0
start stop switch	0	1	0	0	0	0
display action status complete	0	0	0	1	0	0
in-progress	0	0	1	0	0	0
time out	0	0	0	0	0	1
<u>output</u>						
alarm signal	0	0	0	0	1	0
<u>process activation</u>						
monitor and control system	0	1	0	0	1	1
activate/deactivate system	0	1	0	0	0	0
display messages and status	1	0	1	1	1	1
interact with user	1	0	0	1	0	1

The Process Specification(PSPEC) 7.2.4

- used to describe all flow model processes that appear at the final level of refinement
 - narrative text
 - program design language (PDL)
 - mathematical equations
 - tables, or UML activity diagrams.
- To illustrate the use of the PSPEC
 - consider the process password transform represented in the flow model for SafeHome



PSPEC: process password (at control panel)

The process password transform performs password validation at the control panel for the Safe Home security function. Process password receives a four-digit password from the interact with user function. The password is first compared to the master password stored within the system. If the master password matches, <valid id message = true> is passed to the message and status display function. If the master password does not match, the four digits are compared to a table of secondary passwords (these may be assigned to house guests and/or workers who require entry to the home when the owner is not present). If the password matches an entry within the table, <valid id message = true> is passed to the message and status display function. If there is no match, <valid id message = false> is passed to the message and status display function



7.3 Creating A Behavioral Model

- The behavioral model indicates how software will respond to external events or stimuli
- for create this model
 - Evaluate all use cases to fully understand the sequence of interaction within the system.
 - Identify events that drive the interaction sequence and understand how these events relate to specific objects.
 - Create a sequence for each use case.
 - Build a state diagram for the system.
 - Review the behavioral model to verify accuracy and consistency.



7.3.1 Identifying Events with the Use Case

- An event occurs whenever the system and an actor exchange information.
- A use case is examined for points of information exchange. To illustrate, we reconsider the use case for a portion of the *SafeHome security function*.

The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.

- As an example of a typical event, consider the underlined use case phrase “homeowner uses the keypad to key in a four-digit password.”
- It is important to note that some events have an explicit impact on the flow of control of the use case, while others have no direct impact on the flow of control.

State Representations 7.3.2

□ two different characterizations of states must be considered

1. the state of each class as the system performs its function

○ passive

➤ A *passive state* is simply the current status of all of an object's attributes.

○ active

➤ The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing.

2. the state of the system as observed from the outside as the system performs its function



FIGURE 7.6

State diagram
for the
ControlPanel
class

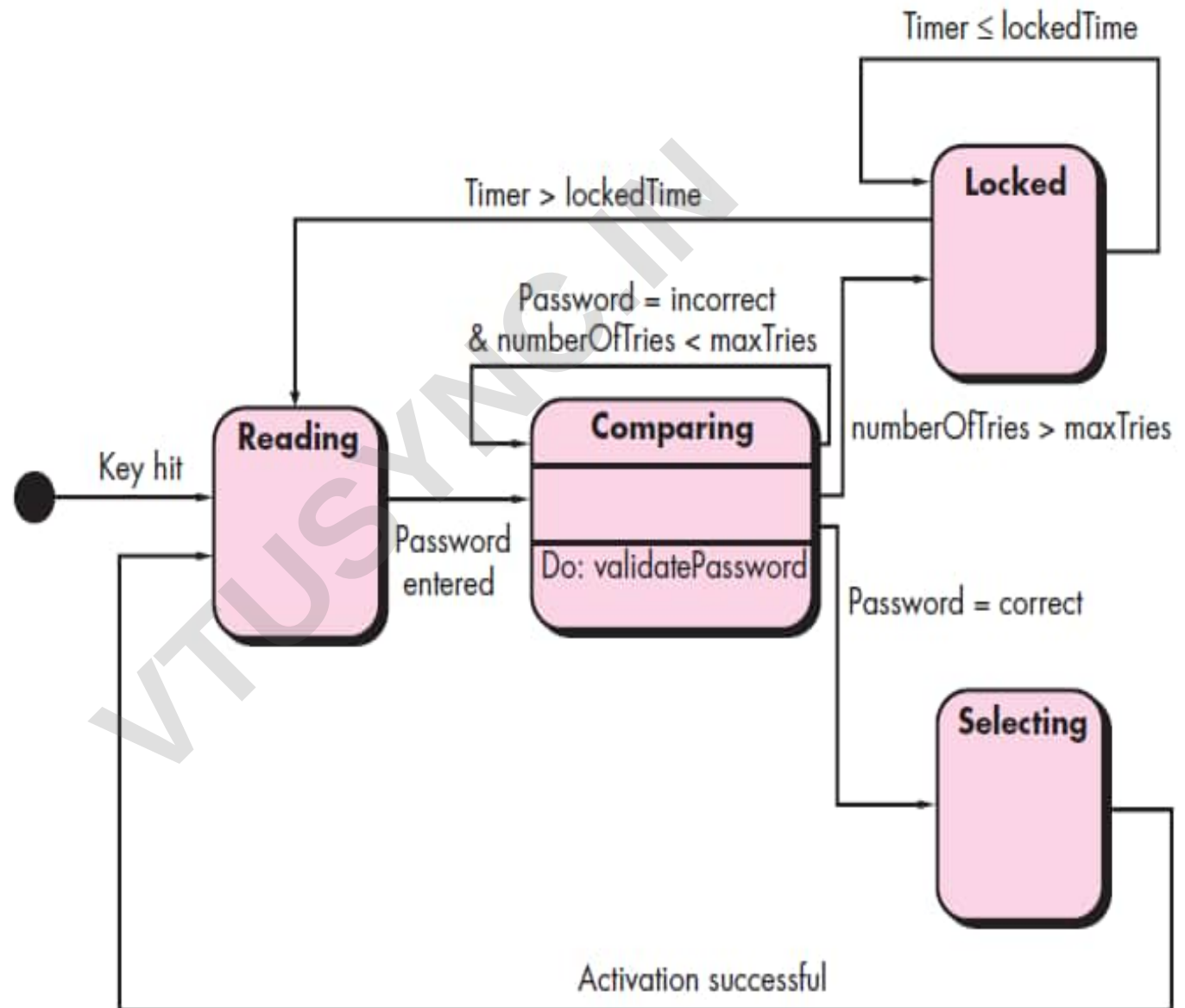
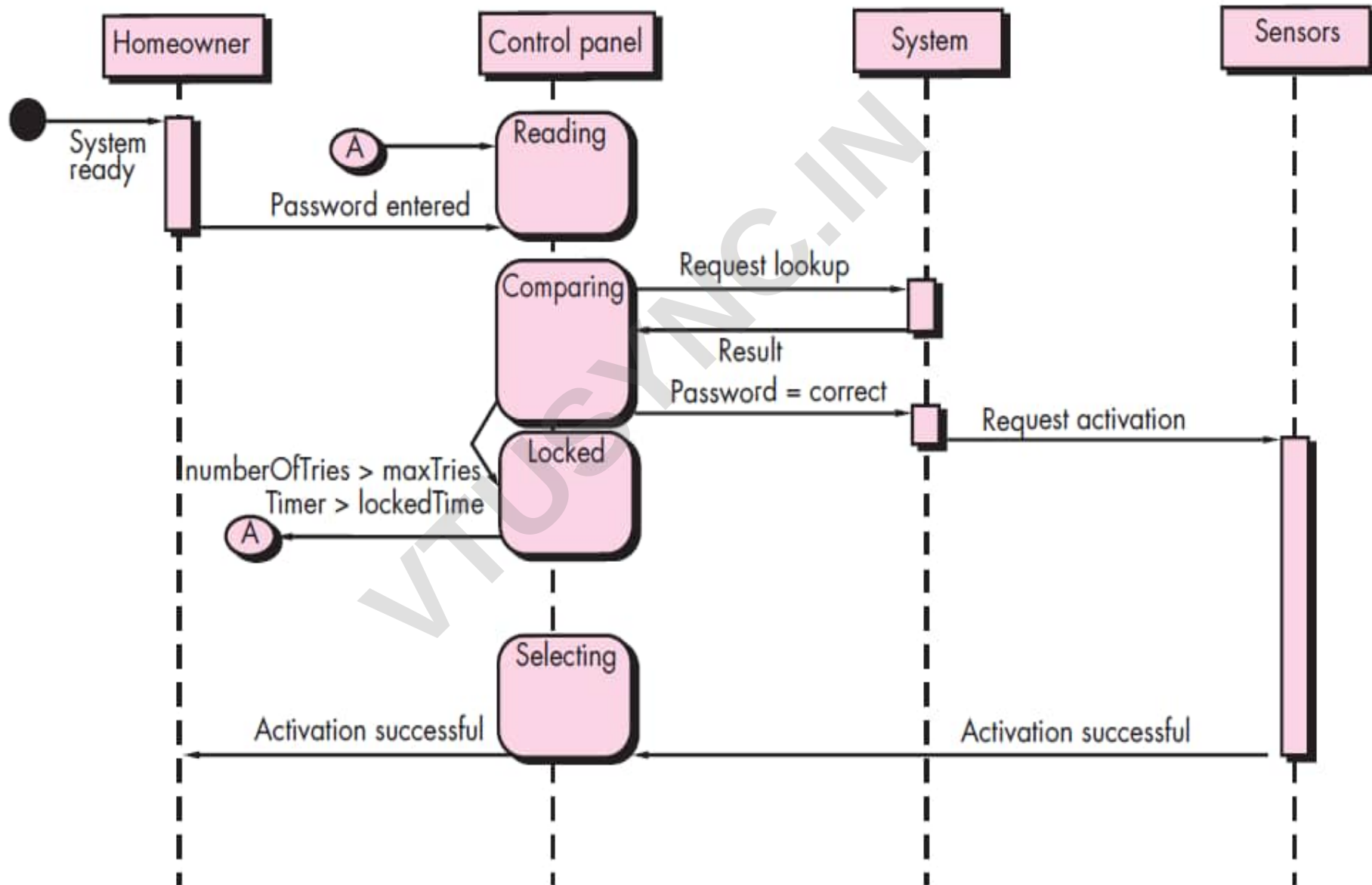


FIGURE 7.7Sequence diagram (partial) for the *SafeHome* security function

Two Different Behavior

State diagrams for analysis classes.

- Indicates how an individual class changes state based on external events
- One component of a behavioral model is a UML state diagram⁹ that represents active states for each class and the events (triggers) that cause changes between these active states

Sequence diagrams.

- shows the behavior of the software as a function of time.
- sequence diagram in UML, indicates how events cause transitions from object to object



The End